



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Creación, Lectura y Escritura de Ficheros
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Nivel - Paralelo
Alumnos participantes:	Guaman Chanahuano Hamilton Alexander, Monar Parco Jhair Alexander, Muyulema Moyolema Mateo Alexander, Villacres Toalombo Josué Alejandro.
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Rubén Caiza
Link del repositorio:	https://github.com/AlexanderrrG/APE05-Ficheros

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades prácticas en el manejo de ficheros mediante la Programación Orientada a Objetos (POO) en C++ y Java, implementando sistemas funcionales de registro, lectura y gestión de datos persistentes que permitan al estudiante comprender el ciclo completo de escritura, lectura y manipulación de archivos de texto como mecanismo de almacenamiento de información.

Específicos:

- Aplicar los conceptos de clases, objetos y métodos en la implementación de sistemas que utilizan ficheros de texto para el almacenamiento y recuperación de datos.
- Implementar operaciones de lectura y escritura de archivos en lenguajes C++ y Java, empleando estructuras de control como ciclos y condicionales para el procesamiento correcto de la información almacenada.
- Construir un sistema CRUD (Crear, Leer, Actualizar, Eliminar) funcional que integre validaciones de datos, manejo de excepciones y persistencia de información en archivos de texto.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.

Codifique en Java cada uno de los ejercicios planteados.

Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios.

Las capturas se subirán en un archivo pdf en el formato de Guías AP

2.5 Listado de equipos, materiales y recursos

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales



- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☒ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Ejercicio 1

Análisis del Problema

El programa debe registrar datos de un estudiante (cédula, nombre, carrera y promedio) y guardarlos en un archivo de texto llamado estudiantes.txt. Se debe confirmar al usuario que el registro fue exitoso. Se aplica POO (clases, objetos, métodos) y escritura de archivos.

Variables de Entrada, Proceso y Salida

- **Entrada:**
 - cedula : string
 - nombre : string
 - carrera : string
 - promedio : float
- **Proceso**
 - Crear objeto Estudiante
 - Abrir archivo en modo append
 - Escribir datos en archivo
 - Cerrar archivo
- **Salida**
 - Mensaje de confirmación en consola
 - Archivo estudiantes.txt con datos guardados

Algoritmo

1. Inicio del programa
2. Solicitar al usuario: cédula, nombre, carrera y promedio
3. Crear un objeto de la clase Estudiante con los datos ingresados
4. Invocar el método guardar() del objeto
5. Dentro de guardar(): abrir el archivo estudiantes.txt en modo escritura (append)
6. Escribir los datos del estudiante separados por comas o salto de línea
7. Cerrar el archivo
8. Mostrar mensaje "Estudiante registrado correctamente"
9. Fin del programa

Pseudocódigo

Algoritmo RegistrarEstudiante
Definir cedula, nombre, carrera Como Cadena
Definir promedio Como Real
Definir archivo Como Entero

Escribir "Ingrese la cedula: "
Leer cedula

Escribir "Ingrese el nombre: "
Leer nombre

Escribir "Ingrese la carrera: "
Leer carrera



Escribir "Ingrese el promedio: "

Leer promedio

archivo <- 1

Abrir archivo, "estudiantes.txt", "Agregar"

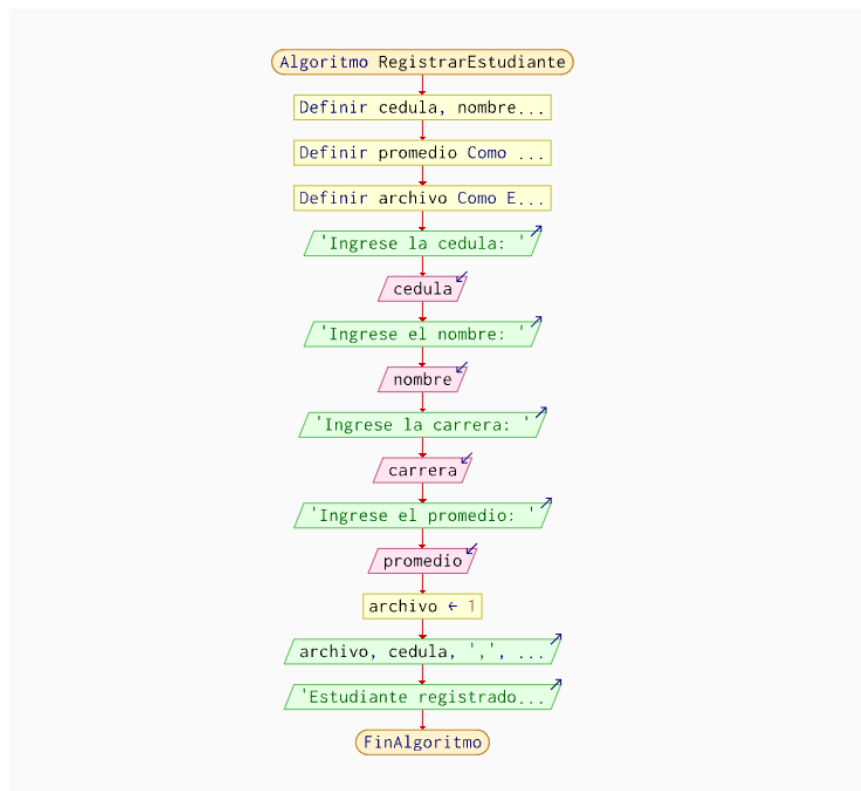
Escribir archivo, cedula, ",", nombre, ",", carrera, ",", promedio

Cerrar archivo

Escribir "Estudiante registrado correctamente."

FinAlgoritmo

Diagrama de Flujo



Código en C++

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Estudiante {
private:
    string cedula, nombre, carrera;
    float promedio;
public:
    Estudiante(string c, string n, string ca, float p)
        : cedula(c), nombre(n), carrera(ca), promedio(p) {}

    void guardar() {
        ofstream archivo("estudiantes.txt", ios::app);
        if (archivo.is_open()) {
            archivo << cedula << "," << nombre << ","
                << carrera << "," << promedio << "\n";
        }
    }
};
```



```
        archivo.close();
        cout << "Estudiante registrado correctamente.\n";
    } else {
        cout << "Error al abrir el archivo.\n";
    }
}
};
```

```
int main() {
    string cedula, nombre, carrera;
    float promedio;
    cout << "Cedula: "; cin >> cedula;
    cin.ignore();
    cout << "Nombre: "; getline(cin, nombre);
    cout << "Carrera: "; getline(cin, carrera);
    cout << "Promedio: "; cin >> promedio;
    Estudiante est(cedula, nombre, carrera, promedio);
    est.guardar();
    return 0;
}
```

Código en Java

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

class Estudiante {
    private String cedula;
    private String nombre;
    private String carrera;
    private double promedio;

    public Estudiante(String cedula, String nombre, String carrera, double promedio) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.carrera = carrera;
        this.promedio = promedio;
    }

    public void guardar() {
        try {
            FileWriter fw = new FileWriter("estudiantes.txt", true);
            PrintWriter pw = new PrintWriter(fw);

            pw.println(cedula + "," + nombre + "," + carrera + "," + promedio);

            pw.close();

            System.out.println("Estudiante registrado correctamente.");
        } catch (IOException e) {
            System.out.println("Error al guardar el archivo.");
        }
    }
}
```



```
}  
  
public class Ejerc2 {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Cedula: ");  
        String cedula = sc.nextLine();  
  
        System.out.print("Nombre: ");  
        String nombre = sc.nextLine();  
  
        System.out.print("Carrera: ");  
        String carrera = sc.nextLine();  
  
        System.out.print("Promedio: ");  
        double promedio = sc.nextDouble();  
  
        Estudiante est = new Estudiante(cedula, nombre, carrera, promedio);  
  
        est.guardar();  
  
        sc.close();  
    }  
}
```

Pruebas de Solución

```
PS C:\C++> cd "c:\C++\" ; if ($?) { g++ Ejerc1.cpp -o Ejerc1 } ; if ($?) { .\Ejerc1 }  
Cedula: 0202441382  
Nombre: Jhair  
Carrera: Software  
Promedio: 7  
Estudiante registrado correctamente.  
PS C:\C++> █
```

Ejecución en C++

```
PS C:\C++> cd "c:\C++\" ; if ($?) { javac Ejerc2.java } ; if ($?) { java Ejerc2 }  
Cedula: 035478254  
Nombre: Lucas  
Carrera: Telecomunicaciones  
Promedio: 8  
Estudiante registrado correctamente.  
PS C:\C++> █
```

Ejecución en Java

Ejercicio 2

Análisis del Problema

Se lee el archivo estudiantes.txt del ejercicio anterior. El programa muestra todos los registros, cuenta cuántos estudiantes hay y calcula el promedio general. Se aplica lectura de archivos, ciclos, arreglos y métodos.



Variables de Entrada, Proceso y Salida

- **Entrada:**
Archivo: estudiantes.txt
línea : Cadena (por ciclo)
- **Proceso:**
Leer línea por línea
Acumular suma de promedios
Contar registros
- **Salida:**
Lista de todos los estudiantes
Total de estudiantes (Entero)
Promedio general (Real)

Algoritmo

1. Inicio del programa
2. Abrir el archivo estudiantes.txt en modo lectura
3. Inicializar contador = 0 y sumaPromedios = 0
4. Mientras no sea fin del archivo: leer línea, mostrar datos, acumular promedio, incrementar contador
5. Cerrar el archivo
6. Mostrar total de estudiantes y promedio general
7. Fin del programa

Pseudocódigo

Algoritmo LeerEstudiantes

Definir cedula, nombre, carrera Como Cadena

Definir promedio, sumaPromedios Como Real

Definir contador Como Entero

Definir respuesta Como Caracter

contador <- 0

sumaPromedios <- 0

respuesta <- "S"

Mientras Mayusculas(respuesta) = "S" Hacer

Escribir "--- Nuevo Estudiante ---"

Escribir "Ingrese la cédula:"

Leer cedula

Escribir "Ingrese el nombre:"

Leer nombre

Escribir "Ingrese la carrera:"

Leer carrera

Escribir "Ingrese el promedio:"

Leer promedio

Escribir "Registro: ", cedula, " | ", nombre, " | ", carrera, " | ", promedio

sumaPromedios <- sumaPromedios + promedio

contador <- contador + 1

Escribir "¿Desea ingresar otro estudiante? (S/N): "

Leer respuesta

FinMientras



Escribir "-----"

Escribir "Total de estudiantes: ", contador

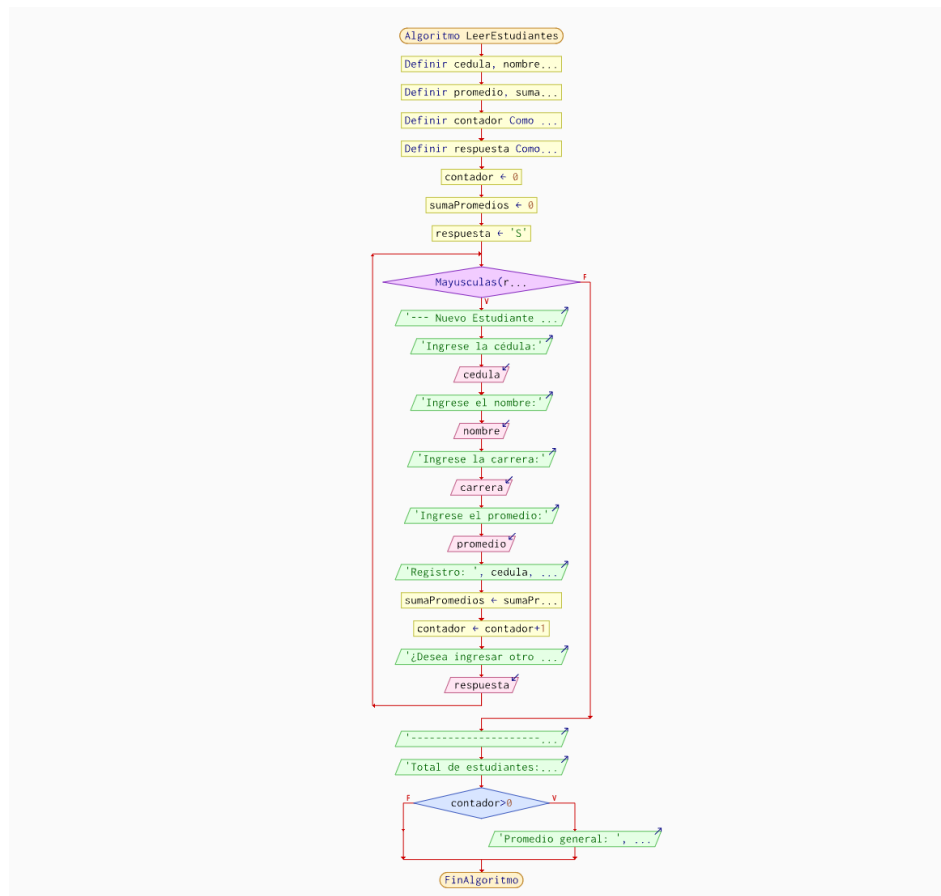
Si contador > 0 Entonces

Escribir "Promedio general: ", sumaPromedios / contador

FinSi

FinAlgoritmo

Diagrama de Flujo



Código en C++

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

class LectorEstudiantes {
public:
    void leerArchivo() {
        ifstream archivo("estudiantes.txt");
        if (!archivo.is_open()) { cout << "No se puede abrir el archivo.\n"; return; }

        string linea; int contador = 0; float suma = 0;
        while (getline(archivo, linea)) {
            stringstream ss(linea);
            string cedula, nombre, carrera, prom;
            getline(ss, cedula, ','); getline(ss, nombre, ',');
```



```
getline(ss, carrera, ','); getline(ss, prom, ',');
cout << cedula << " | " << nombre << " | " << carrera << " | " <<
prom << "\n";
suma += stof(prom); contador++;
}
archivo.close();
cout << "\nTotal: " << contador << "\n";
if (contador > 0) cout << "Promedio general: " << suma / contador <<
"\n";
}
};

int main() { LectorEstudiantes l; l.leerArchivo(); return 0; }
```

Código en Java

```
import java.io.*;

class LectorEstudiantes {
    public void leerArchivo() {
        try (BufferedReader br = new BufferedReader(new
        FileReader("estudiantes.txt"))) {
            String linea; int contador = 0; double suma = 0;
            while ((linea = br.readLine()) != null) {
                String[] c = linea.split(",");
                System.out.println(c[0] + " | " + c[1] + " | " + c[2] + " | " + c[3]);
                suma += Double.parseDouble(c[3]); contador++;
            }
            System.out.println("\nTotal: " + contador);
            if (contador > 0) System.out.printf("Promedio general: %.2f\n", suma /
            contador);
        } catch (IOException e) { System.out.println("Error: " + e.getMessage());
        }
    }
}

public class Ejerc2 {
    public static void main(String[] args) { new
    LectorEstudiantes().leerArchivo(); }
}
```

Pruebas de Solución

```
PS C:\C++> cd "c:\C++\" ; if ($?) { g++ Ejerc2.cpp -o Ejerc2 } ; if ($?) { .\Ejerc2 }
0202441382 | Jhair | Software | 7
035478254 | Lucas | Telecomunicaciones | 8.0

Total: 2
Promedio general: 7.5
```

Ejecución en C++

```
PS C:\C++> cd "c:\C++\" ; if ($?) { javac Ejerc2.java } ; if ($?) { java Ejerc2 }
0202441382 | Jhair | Software | 7
035478254 | Lucas | Telecomunicaciones | 8.0

Total: 2
Promedio general: 7,50
```

Ejecución en Java



Análisis del Problema

Sistema para registrar productos con código, nombre, cantidad y precio en inventario.txt. Se calcula el valor total del inventario (cantidad × precio por cada producto y acumulado general). Se aplica POO, archivos y métodos.

Variables de Entrada, Proceso y Salida

- **Entrada:**
codigo : Cadena
nombre : Cadena
cantidad : Entero
precio : Real
- **Proceso:**
valorProducto = cantidad * precio
Guardar en inventario.txt
Leer y acumular valorTotal
- **Salida:**
Confirmación de guardado con valor del producto
Valor total del inventario (Real)

Algoritmo

1. Solicitar código, nombre, cantidad y precio
2. Calcular valorProducto = cantidad × precio
3. Abrir inventario.txt en modo append y escribir los datos junto al valor del producto
4. Cerrar el archivo y confirmar guardado
5. Abrir inventario.txt en lectura para calcular el valor total del inventario
6. Mientras haya líneas: acumular el campo valorProducto
7. Mostrar valor total del inventario

Pseudocódigo

Algoritmo SistemaInventario

Definir codigo, nombre Como Cadena

Definir cantidad Como Entero

Definir precio, valorProducto, valorTotal Como Real

Definir respuesta Como Caracter

valorTotal <- 0

respuesta <- "S"

Mientras Mayusculas(respuesta) = "S" Hacer

Escribir "--- Registro de Producto ---"

Escribir "Codigo del producto: "

Leer codigo

Escribir "Nombre del producto: "

Leer nombre

Escribir "Cantidad: "

Leer cantidad

Escribir "Precio: "

Leer precio

*valorProducto <- cantidad * precio*

Escribir "Producto guardado en memoria. Valor: \$", valorProducto

valorTotal <- valorTotal + valorProducto

Escribir "¿Desea registrar otro producto? (S/N): "

Leer respuesta



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026

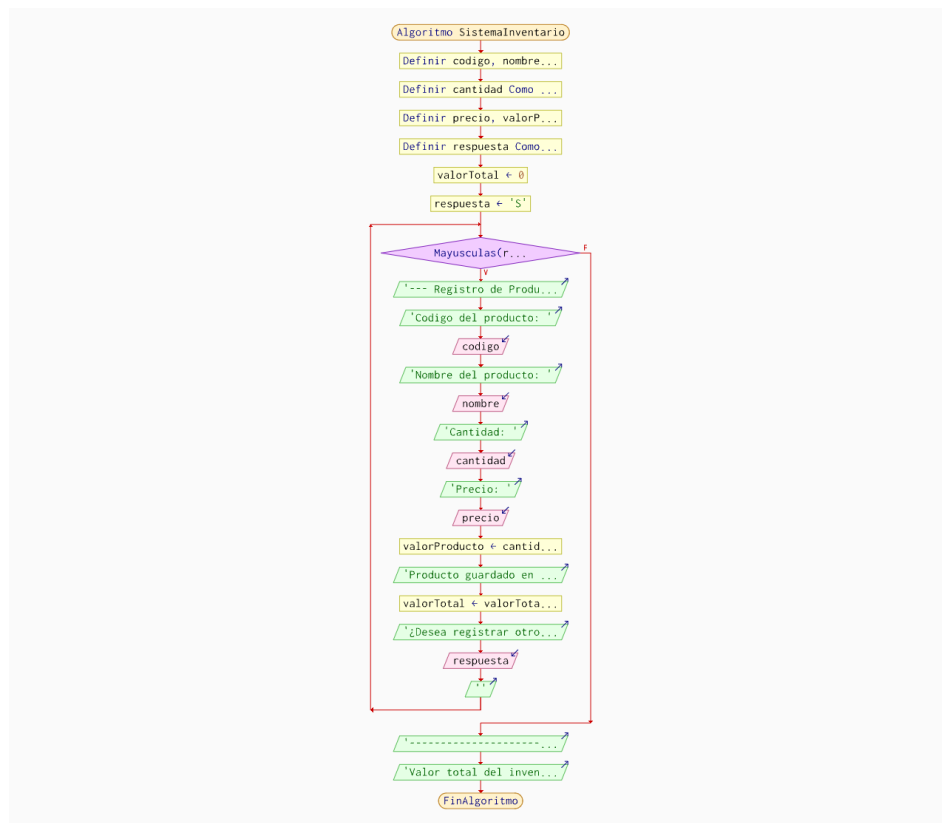


Escribir ""
FinMientras

Escribir "-----"
Escribir "Valor total del inventario: \$", valorTotal

FinAlgoritmo

Diagrama de Flujo



Código en C++

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

class Producto {
private:
    string codigo, nombre; int cantidad; float precio;
public:
    Producto(string co, string n, int ca, float p)
        : codigo(co), nombre(n), cantidad(ca), precio(p) {}

    void guardar() {
        float valor = cantidad * precio;
        ofstream f("inventario.txt", ios::app);
        if (f.is_open()) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
f << codigo << "," << nombre << "," << cantidad << "," << precio
<< "," << valor << "\n";
f.close();
cout << "Guardado. Valor del producto: $" << valor << "\n";
}
}

void calcularValorTotal() {
    ifstream f("inventario.txt");
    string linea; float total = 0;
    while (getline(f, linea)) {
        stringstream ss(linea); string campo; int i = 0;
        while (getline(ss, campo, ',')) { if (i == 4) total += stof(campo); i++; }
    }
    cout << "Valor total inventario: $" << total << "\n";
}
};

int main() {
    string codigo, nombre; int cantidad; float precio;
    cout << "Codigo: "; cin >> codigo; cin.ignore();
    cout << "Nombre: "; getline(cin, nombre);
    cout << "Cantidad: "; cin >> cantidad;
    cout << "Precio: "; cin >> precio;
    Producto p(codigo, nombre, cantidad, precio);
    p.guardar(); p.calcularValorTotal();
    return 0;
}
```

Código en Java

```
import java.io.*; import java.util.Scanner;

class Producto {
    private String codigo, nombre; private int cantidad; private double precio;
    public Producto(String co, String n, int ca, double p) { codigo=co;
    nombre=n; cantidad=ca; precio=p; }

    public void guardar() {
        double valor = cantidad * precio;
        try (PrintWriter pw = new PrintWriter(new FileWriter("inventario.txt",
        true))) {
            pw.println(codigo+","+nombre+","+cantidad+","+precio+","+valor);
            System.out.printf("Guardado. Valor: $%.2f%n", valor);
        } catch (IOException e) { System.out.println(e.getMessage()); }
    }

    public void calcularValorTotal() {
        double total = 0;
        try (BufferedReader br = new BufferedReader(new
        FileReader("inventario.txt"))) {
            String l; while ((l = br.readLine()) != null) total +=
            Double.parseDouble(l.split(",")[4]);
            System.out.printf("Valor total: $%.2f%n", total);
        } catch (IOException e) { System.out.println(e.getMessage()); }
    }
}
```



```
}  
public class Ejerc3 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Codigo: "); String co = sc.next(); sc.nextLine();  
        System.out.print("Nombre: "); String n = sc.nextLine();  
        System.out.print("Cantidad: "); int ca = sc.nextInt();  
        System.out.print("Precio: "); double p = sc.nextDouble();  
        Producto prod = new Producto(co, n, ca, p);  
        prod.guardar(); prod.calcularValorTotal();  
    }  
}
```

Prueba de Solución

```
PS C:\C++> cd "c:\C++\" ; if ($?) { g++ Ejerc3.cpp -o Ejerc3 } ; if ($?) { .\Ejerc3 }  
Codigo: 0256  
Nombre: Jabon  
Cantidad: 5  
Precio: 1.25  
Guardado. Valor del producto: $6.25  
Valor total inventario: $6.25
```

Ejecución en C++

```
Codigo: 0356  
Nombre: Shampoo  
Cantidad: 3  
Precio: 3,50  
Guardado. Valor: $10,50  
Valor total: $16,75
```

Ejecución en Java

Ejercicio 4

Análisis del Problema

Agenda telefónica que guarda contactos (nombre, teléfono, correo) en agenda.txt.
Permite agregar nuevos contactos, buscar por nombre y mostrar todos los registrados.
Se aplica archivos, búsqueda lineal y métodos.

Variables de Entrada, Proceso y Salida

- **Entrada:**
 - nombre : Cadena
 - telefono : Cadena
 - correo : Cadena
 - opcion : Entero
 - busqueda : Cadena
- **Proceso:**
 - Agregar: escribir en agenda.txt
 - Buscar: leer y comparar nombre
 - Mostrar: leer todo el archivo
- **Salida:**
 - Confirmación al agregar
 - Datos del contacto buscado (o "no encontrado")
 - Lista completa de contactos

Algoritmo

1. Inicio
2. Mostrar menú: 1-Agregar, 2-Buscar, 3-Mostrar todos, 4-Salir
3. Leer opción del usuario
4. Si opción = 1: pedir nombre, teléfono, correo → guardar en agenda.txt
5. Si opción = 2: pedir nombre → leer archivo y comparar cada registro



6. Si opción = 3: leer y mostrar todos los registros de agenda.txt
7. Repetir hasta que el usuario elija opción 4
8. Fin

Pseudocódigo

Algoritmo AgendaTelefonica

Dimension nombres[100], telefonos[100], correos[100]

Definir busqueda Como Cadena

Definir opcion, totalContactos, i Como Entero

Definir encontrado Como Logico

totalContactos <- 0

Repetir

Escribir ""

Escribir "1-Agregar 2-Buscar 3-Mostrar todos 4-Salir"

Escribir "Seleccione una opcion: "

Leer opcion

Segun opcion Hacer

1:

Si totalContactos < 100 Entonces

totalContactos <- totalContactos + 1

Escribir "Nombre: "

Leer nombres[totalContactos]

Escribir "Telefono: "

Leer telefonos[totalContactos]

Escribir "Correo: "

Leer correos[totalContactos]

Escribir "Contacto agregado correctamente."

Sino

Escribir "La agenda está llena (límite de 100)."

FinSi

2:

Escribir "Nombre a buscar: "

Leer busqueda

encontrado <- Falso

Para i <- 1 Hasta totalContactos Hacer

Si nombres[i] = busqueda Entonces

Escribir nombres[i], " | ", telefonos[i], " | ", correos[i]

encontrado <- Verdadero

FinSi

FinPara

Si NO encontrado Entonces

Escribir "Contacto no encontrado."

FinSi

3:

Escribir "--- Todos los contactos ---"

Si totalContactos = 0 Entonces

Escribir "La agenda está vacía."

Sino

Para i <- 1 Hasta totalContactos Hacer

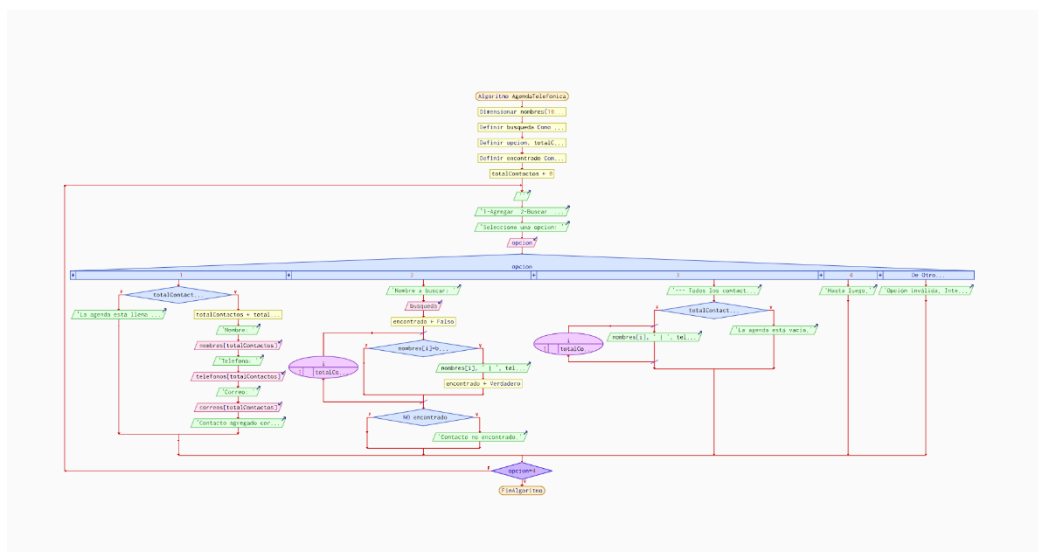


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir nombres[i], " | ", telefonos[i], " | ", correos[i]
 FinPara
 FinSi
 4:
 Escribir "Hasta luego."
 De Otro Modo:
 Escribir "Opción inválida. Intente de nuevo."
 FinSegun
 Hasta Que opcion = 4

FinAlgoritmo
Diagrama de Flujo



Código en C++

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

class Agenda {
public:
    void agregar(string nombre, string tel, string correo) {
        ofstream f("agenda.txt", ios::app);
        if (f.is_open()) { f << nombre << " | " << tel << " | " << correo << "\n";
        f.close(); }
        cout << "Contacto agregado.\n";
    }

    void buscar(string busq) {
        ifstream f("agenda.txt"); string linea; bool ok = false;
        while (getline(f, linea)) {
            stringstream ss(linea); string n, t, c;
            getline(ss, n, '|'); getline(ss, t, '|'); getline(ss, c, '|');
            if (n == busq) { cout << n << " | " << t << " | " << c << "\n"; ok =
true; }
        }
    }
};
```



```
        if (!ok) cout << "Contacto no encontrado.\n";
    }
    void mostrarTodos() {
        ifstream f("agenda.txt"); string l;
        cout << "--- Contactos ---\n";
        while (getline(f, l)) cout << l << "\n";
    }
};

int main() {
    Agenda ag; int op;
    do {
        cout << "\n1-Agregar 2-Buscar 3-Mostrar 4-Salir: "; cin >> op;
        cin.ignore();
        if (op==1) { string n,t,c;
            cout<<"Nombre: "; getline(cin,n); cout<<"Telefono: "; getline(cin,t);
            cout<<"Correo: "; getline(cin,c);
            ag.agregar(n,t,c);
        } else if (op==2) { string b; cout<<"Buscar: "; getline(cin,b);
            ag.buscar(b); }
        else if (op==3) ag.mostrarTodos();
    } while (op!=4);
    return 0;
}
```

Código en Java

```
import java.io.*; import java.util.Scanner;

class Agenda {
    public void agregar(String n, String t, String c) {
        try (PrintWriter pw = new PrintWriter(new FileWriter("agenda.txt",true)))
        {
            pw.println(n+" "+t+" "+c); System.out.println("Contacto agregado.");
        } catch (IOException e) { System.out.println(e.getMessage()); }
    }
    public void buscar(String busq) {
        boolean ok = false;
        try (BufferedReader br = new BufferedReader(new
        FileReader("agenda.txt"))) {
            String l;
            while ((l = br.readLine()) != null) {
                String[] c = l.split(" ");
                if (c[0].equalsIgnoreCase(busq)) { System.out.println(c[0]+" |
                "+c[1]+" | "+c[2]); ok=true; }
            }
        } catch (IOException e) { System.out.println(e.getMessage()); }
        if (!ok) System.out.println("Contacto no encontrado.");
    }
    public void mostrarTodos() {
        System.out.println("--- Contactos ---");
        try (BufferedReader br = new BufferedReader(new
        FileReader("agenda.txt"))) {
            String l; while ((l=br.readLine())!=null) System.out.println(l);
        } catch (IOException e) { System.out.println(e.getMessage()); }
    }
}
```



```
}  
}  
}  
public class Main {  
    public static void main(String[] args) {  
        Agenda ag = new Agenda(); Scanner sc = new Scanner(System.in); int op;  
        do {  
            System.out.print("\n1-Agregar 2-Buscar 3-Mostrar 4-Salir: ");  
            op=sc.nextInt(); sc.nextLine();  
            switch(op){  
                case 1: System.out.print("Nombre: "); String n=sc.nextLine();  
                    System.out.print("Telefono: "); String t=sc.nextLine();  
                    System.out.print("Correo: "); String c=sc.nextLine();  
                    ag.agregar(n,t,c); break;  
                case 2: System.out.print("Buscar: "); ag.buscar(sc.nextLine()); break;  
                case 3: ag.mostrarTodos(); break;  
            }  
        } while(op!=4);  
    }  
}
```

Prueba de Solución

```
1-Agregar 2-Buscar 3-Mostrar 4-Salir: 1  
Nombre: Jhair  
Telefono: 0967396119  
Correo: jhairmonar41@gmail.com  
Contacto agregado.  
  
1-Agregar 2-Buscar 3-Mostrar 4-Salir: 2  
Buscar: Jhair  
Jhair | 0967396119 | jhairmonar41@gmail.com  
  
1-Agregar 2-Buscar 3-Mostrar 4-Salir: █
```

Ejecución en C++

```
1-Agregar 2-Buscar 3-Mostrar 4-Salir: 1  
Nombre: Jhair  
Telefono: 097725263  
Correo: jahirhkaja@  
Contacto agregado.  
  
1-Agregar 2-Buscar 3-Mostrar 4-Salir: 3  
--- Contactos ---  
Jhair | 0967396119 | jhairmonar41@gmail.com  
Jhair | 097725263 | jahirhkaja@  
  
1-Agregar 2-Buscar 3-Mostrar 4-Salir: █
```

Ejecución en Java

Ejercicio 5

1. Análisis del Problema

El objetivo es desarrollar un sistema para registrar calificaciones de estudiantes, calcular promedios, determinar si el alumno está aprobado o reprobado y guardar el registro en un archivo de texto utilizando POO.

Incluye entradas con Nombre del estudiante (texto), nota 1 (decimal), nota 2 (decimal) y nota 3 (decimal).

Procesos:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



1. Validar las notas ingresadas por el estudiante.
2. Calcular el promedio sumando las tres notas y dividiendo el resultado para 3.
3. Determinar la condición de aprobado o reprobado comparando el promedio con la nota mínima requerida.
4. Instanciar un objeto de la clase Estudiante para empaquetar los atributos y métodos correspondientes.
5. Guardar los datos del estudiante, su promedio y su estado en el archivo notas_estudiantes.txt.

En las salidas estará la visualización de los datos procesados en la consola y el almacenamiento permanente de la información dentro del archivo .txt.

2. Variables de Entrada, Proceso y Salida

Entrada

- nombre (Texto / String): El nombre completo del estudiante que se va a registrar.
- nota1 (Decimal / Double o Float): La primera calificación del estudiante.
- nota2 (Decimal / Double o Float): La segunda calificación del estudiante.
- nota3 (Decimal / Double o Float): La tercera calificación del estudiante.

Proceso

- promedio (Decimal / Double o Float): Variable interna que almacena el resultado de sumar las tres notas y dividir las entre tres.
- condicion (Texto / String): Almacena la palabra "Aprobado" o "Reprobado" tras evaluar analíticamente el promedio obtenido.
- estudiante (Objeto de la clase Estudiante): Instancia que empaqueta los atributos (nombre y notas) y los métodos (calcular promedio y estado).
- archivo (Flujo de archivo / Objeto de escritura): Elemento lógico que se conecta con el sistema operativo para abrir, escribir y cerrar el documento de texto en el disco duro.

Salida

- Registro físico (Archivo de texto .txt): Documento guardado en la computadora que contiene las líneas de texto con el nombre, las notas, el promedio calculado y la condición final del estudiante.
- Mensaje de confirmación: Texto en pantalla que avisa al usuario que los datos se procesaron y guardaron con éxito.

3. Algoritmo (Paso a paso)

1. Inicio.
2. Definir la estructura de un "Estudiante" con los espacios necesarios para su nombre, sus tres notas, su promedio y su estado final.
3. Mostrar un mensaje en pantalla solicitando al usuario que ingrese el nombre del estudiante.
4. Guardar el nombre ingresado.
5. Mostrar mensajes en pantalla para solicitar de forma consecutiva la primera, la segunda y la tercera nota.
6. Guardar cada una de las tres notas ingresadas por el usuario.
7. Asignar los datos recopilados (nombre y notas) al objeto del estudiante.
8. Calcular el promedio sumando las tres notas guardadas en el estudiante y dividiendo ese resultado exacto entre tres.



9. Evaluar el promedio obtenido:
 - Si el promedio es igual o mayor a la nota mínima de aprobación, registrar que la condición del estudiante es "Aprobado".
 - Si el promedio es menor a la nota mínima, registrar que la condición del estudiante es "Reprobado".
10. Intentar abrir o crear un archivo de texto en la computadora destinado a almacenar el registro de notas.
11. Escribir dentro del archivo de texto toda la información organizada: el nombre del alumno, sus tres notas individuales, el promedio final calculado y si aprobó o reprobó.
12. Cerrar el archivo de texto de forma segura para garantizar que los datos se guarden correctamente en el disco duro.
13. Mostrar un mensaje en pantalla confirmando al usuario que el estudiante ha sido registrado y los datos se han salvado en el archivo.
14. Fin.

4.Pseudocodigo

Clase Estudiante

// Atributos privados

Privado Cadena nombre

Privado Real nota1, nota2, nota3

Privado Real promedio

Privado Cadena condicion

// Constructor

Publico Procedimiento Estudiante(Cadena nom, Real n1, Real n2, Real n3)

nombre <- nom

nota1 <- n1

nota2 <- n2

nota3 <- n3

promedio <- 0.0

condicion <- ""

Fin Procedimiento

// Métodos

Publico Procedimiento calcularPromedio()

promedio <- (nota1 + nota2 + nota3) / 3

Fin Procedimiento

Publico Procedimiento determinarCondicion()

Si promedio >= 7.0 Entonces

condicion <- "Aprobado"

Sino

condicion <- "Reprobado"

Fin Si

Fin Procedimiento



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



// Métodos para obtener los datos (Getters)

Publico Funcion obtenerNombre() : Cadena

Devolver nombre

Fin Funcion

Publico Funcion obtenerPromedio() : Real

Devolver promedio

Fin Funcion

Publico Funcion obtenerCondicion() : Cadena

Devolver condicion

Fin Funcion

Fin Clase

Algoritmo Sistema_Notas

Variables:

Cadena nom

Real n1, n2, n3

Estudiante est

Archivo file

Inicio

Escribir "Ingrese el nombre del estudiante: "

Leer nom

Escribir "Ingrese la nota 1: "

Leer n1

Escribir "Ingrese la nota 2: "

Leer n2

Escribir "Ingrese la nota 3: "

Leer n3

// Crear el objeto

est <- Nuevo Estudiante(nom, n1, n2, n3)

// Procesar datos mediante sus métodos

est.calcularPromedio()

est.determinarCondicion()

// Guardar en archivo de texto

Abrir Archivo "notas_estudiantes.txt" en Modo Escritura/Anexar como file

Si file está abierto Entonces

Escribir_En_Archivo file, "Nombre: " + est.obtenerNombre()

Escribir_En_Archivo file, "Promedio: " + est.obtenerPromedio()



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir_En_Archivo file, "Condicion: " + est.obtenerCondicion()

Escribir_En_Archivo file, "-----"

Cerrar Archivo file

Escribir "Datos guardados en el archivo correctamente."

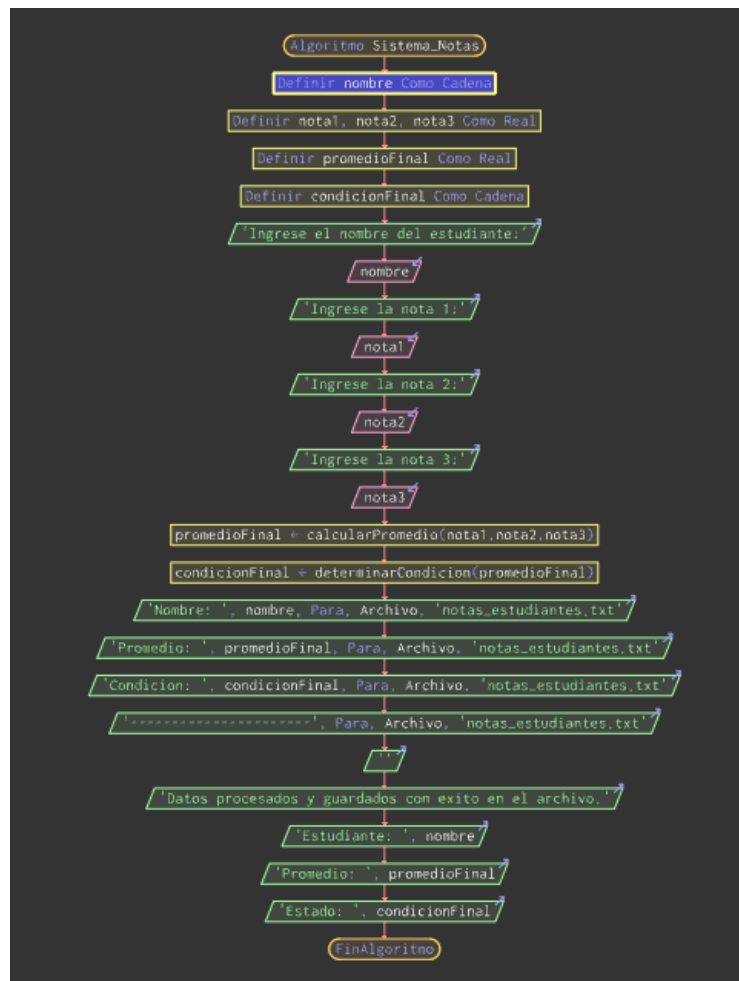
Sino

Escribir "Error al abrir el archivo."

Fin Si

Fin

5. Diagrama de Flujo



6. Código en C++

```
#include <iostream>
#include <string>
#include <fstream> // Librería para el manejo de archivos

using namespace std;

class Estudiante {
private:
    string nombre;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
double nota1, nota2, nota3;  
double promedio;  
string condicion;
```

```
public:
```

```
// Constructor
```

```
Estudiante(string nom, double n1, double n2, double n3) {  
    nombre = nom;  
    nota1 = n1;  
    nota2 = n2;  
    nota3 = n3;  
    promedio = 0.0;  
    condicion = "";  
}
```

```
// Métodos de proceso
```

```
void calcularPromedio() {  
    promedio = (nota1 + nota2 + nota3) / 3.0;  
}
```

```
void determinarCondicion() {  
    if (promedio >= 7.0) {  
        condicion = "Aprobado";  
    } else {  
        condicion = "Reprobado";  
    }  
}
```

```
// Métodos de acceso (Getters)
```

```
string getNombre() { return nombre; }  
double getPromedio() { return promedio; }  
string getCondicion() { return condicion; }  
};
```

```
int main() {
```

```
    string nom;  
    double n1, n2, n3;
```

```
    cout << "Ingrese el nombre del estudiante: ";  
    getline(cin, nom);
```

```
    cout << "Ingrese la nota 1: ";  
    cin >> n1;  
    cout << "Ingrese la nota 2: ";
```



```
cin >> n2;
cout << "Ingrese la nota 3: ";
cin >> n3;

// Crear la instancia del objeto POO
Estudiante est(nom, n1, n2, n3);
est.calcularPromedio();
est.determinarCondicion();

// Guardar los datos en un archivo (Modo ios::app para no borrar los anteriores)
ofstream archivo("notas_estudiantes.txt", ios::app);

if (archivo.is_open()) {
    archivo << "Nombre: " << est.getNombre() << "\n";
    archivo << "Promedio: " << est.getPromedio() << "\n";
    archivo << "Condicion: " << est.getCondicion() << "\n";
    archivo << "-----\n";
    archivo.close(); // Cerrar el archivo de forma segura
    cout << "¡Datos guardados con exito en 'notas_estudiantes.txt'!" << endl;
} else {
    cout << "Error: No se pudo abrir o crear el archivo." << endl;
}

return 0;
}
```

7.Codigo en Java

```
import java.util.Scanner;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.io.IOException;

// Definición de la clase Estudiante (POO)
class Estudiante {
    private String nombre;
    private double nota1, nota2, nota3;
    private double promedio;
    private String condicion;

    // Constructor
    public Estudiante(String nombre, double nota1, double nota2, double nota3) {
        this.nombre = nombre;
        this.nota1 = nota1;
        this.nota2 = nota2;
        this.nota3 = nota3;
    }
}
```



```
}

// Métodos de negocio
public void calcularPromedio() {
    this.promedio = (nota1 + nota2 + nota3) / 3.0;
}

public void determinarCondicion() {
    if (this.promedio >= 7.0) {
        this.condicion = "Aprobado";
    } else {
        this.condicion = "Reprobado";
    }
}

// Getters
public String getNombre() { return nombre; }
public double getPromedio() { return promedio; }
public String getCondicion() { return condicion; }
}

// Clase Principal
public class Main {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Ingrese el nombre del estudiante: ");
        String nom = teclado.nextLine();

        System.out.print("Ingrese la nota 1: ");
        double n1 = teclado.nextDouble();
        System.out.print("Ingrese la nota 2: ");
        double n2 = teclado.nextDouble();
        System.out.print("Ingrese la nota 3: ");
        double n3 = teclado.nextDouble();

        // Instanciar el objeto
        Estudiante est = new Estudiante(nom, n1, n2, n3);
        est.calcularPromedio();
        est.determinarCondicion();

        // Guardar la información en el archivo de texto
        // El valor 'true' en FileWriter habilita el modo "Append" (anexar sin borrar)
        try (FileWriter archivo = new FileWriter("notas_estudiantes.txt", true);
```



```
PrintWriter escritor = new PrintWriter(archivo)) {

    escritor.println("Nombre: " + est.getNombre());
    escritor.println("Promedio: " + est.getPromedio());
    escritor.println("Condicion: " + est.getCondicion());
    escritor.println("-----");

    System.out.println("Datos guardados con exito en 'notas_estudiantes.txt'!");

} catch (IOException e) {
    System.out.println("Ocurrio un error al intentar escribir en el archivo: " +
e.getMessage());
}

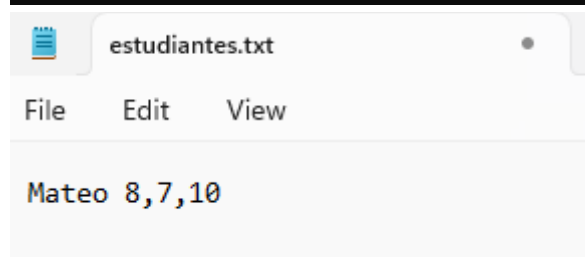
    teclado.close();
}
}
```

8.Pruebas de Solución en C++

```
Ingrese el nombre del estudiante: Mateo
Ingrese la nota 1: 8
Ingrese la nota 2: 7
Ingrese la nota 3: 10

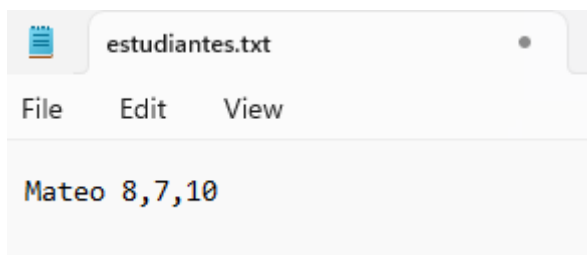
¡Datos guardados con exito en 'notas_estudiantes.txt'!

Process returned 0 (0x0)   execution time : 11.465 s
Press any key to continue.
```



9.Prueba de Solución en Java

```
Ingrese el nombre del estudiante: Mateo
Ingrese la nota 1: 8
Ingrese la nota 2: 7
Ingrese la nota 3: 10
Datos guardados con exito en 'notas_estudiantes.txt'!
-----
BUILD SUCCESS
```

Ejercicio 6

1. Análisis del Problema

El objetivo es desarrollar un sistema para el control de asistencia de estudiantes, mostrar el historial acumulado en pantalla, contar las faltas totales y guardar la información en un archivo de texto utilizando ciclos y validaciones de datos.

Incluye entradas con Nombre del estudiante (texto), fecha del registro (texto) y estado de la asistencia (carácter o texto limitado a 'A' o 'F').

Procesos:

1. Validar mediante un ciclo repetitivo que el estado ingresado sea únicamente 'A' para Asistió o 'F' para Faltó.
2. Traducir el carácter validado a la palabra completa correspondiente ("Asistio" o "Falto").
3. Guardar el nombre, la fecha y el estado de asistencia de forma secuencial en el archivo asistencia.txt.
4. Leer el archivo de texto línea por línea para recuperar los registros guardados.
5. Contar de manera acumulativa cada vez que se detecte una inasistencia en el historial.

En las salidas estará el menú interactivo para el usuario, la impresión del historial completo de asistencias en la consola y la visualización del total de faltas contabilizadas desde el archivo .txt.

2. Variables de Entrada, Proceso y Salida

Entrada

- opcion (Entero / Int): Almacena la elección del usuario dentro del menú interactivo (1 para Registrar, 2 para Historial, 3 para Salir).
- nombre (Texto / String): Nombre completo del estudiante al que se le toma asistencia.
- fecha (Texto / String): La fecha del registro (ej. "22/05/2026").
- estadoIngresado (Texto o Carácter): Letra ingresada por el usuario para definir la asistencia ('A' o 'F').

Proceso

- estadoFinal (Texto / String): Traduce la letra ingresada a la palabra completa ("Asistio" o "Falto") antes de guardarla.
- totalFaltas (Entero / Int): Contador acumulativo que se incrementa en 1 cada vez que, al leer el archivo histórico, se detecta que un registro contiene la palabra "Falto".
- lineaLeida (Texto / String): Almacena temporalmente cada fila de texto recuperada desde el archivo durante la lectura del historial.

Salida



- Registro físico (Archivo de texto "asistencia.txt"): Documento donde se guardan de forma permanente las líneas con los datos de asistencia.
- Reporte en pantalla: Muestra el listado de todo el historial guardado y el número total de faltas contabilizadas.

3. Algoritmo (Paso a paso)

1. Inicio.
2. Definir e inicializar las variables necesarias para controlar el menú, los datos del estudiante y el contador de faltas en cero.
3. Iniciar un ciclo repetitivo que muestre el menú principal en pantalla con las opciones:
1. Registrar Asistencia, 2. Mostrar Historial y Contar Faltas, 3. Salir.
4. Leer la opción seleccionada por el usuario.
5. Si la opción es 1 (Registrar):
 - Solicitar e ingresar el nombre del estudiante y la fecha.
 - Iniciar un ciclo de validación: Solicitar el estado ("A" para Asistió, "F" para Faltó). Si el usuario ingresa una letra diferente, repetir la pregunta hasta que la entrada sea correcta.
 - Asignar la palabra completa "Asistio" o "Falto" según corresponda.
 - Abrir el archivo "asistencia.txt" en modo de añadir texto.
 - Escribir la línea formateada con el nombre, fecha y estado, y cerrar el archivo.
6. Si la opción es 2 (Mostrar Historial):
 - Reiniciar el contador de faltas a cero.
 - Intentar abrir el archivo "asistencia.txt" en modo de lectura.
 - Si el archivo existe, leerlo línea por línea:
 - Mostrar cada línea en pantalla.
 - Si la línea contiene la palabra "Falto", sumar 1 al contador de faltas.
 - Cerrar el archivo.
 - Mostrar en pantalla el total de faltas contabilizadas.
7. Si la opción es 3 (Salir):
 - Mostrar un mensaje de despedida y terminar el ciclo principal.
8. Si la opción no es válida, avisar al usuario y volver a mostrar el menú.
9. Fin.

4. Pseudocódigo

Algoritmo Control_Asistencia

Definir opcion Como Entero

Definir nombre, fecha, estadoIngresado, estadoFinal Como Cadena

Repetir

 Escribir " "

 Escribir "=== CONTROL DE ASISTENCIA ==="

 Escribir "1. Registrar Asistencia"

 Escribir "2. Mostrar Historial"

 Escribir "3. Salir"

 Escribir "Seleccione una opcion:"

 Leer opcion



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Segun opcion Hacer

1:

Escribir "Ingrese nombre del estudiante:"

Leer nombre

Escribir "Ingrese fecha (DD/MM/AAAA):"

Leer fecha

Repetir

Escribir "Ingrese estado (A = Asistio, F = Falto):"

Leer estadoIngresado

Si estadoIngresado <> "A" Y estadoIngresado <> "F" Entonces

Escribir "Error: Estado invalido."

FinSi

Hasta Que estadoIngresado = "A" O estadoIngresado = "F"

Si estadoIngresado = "A" Entonces

estadoFinal <- "Asistio"

Sino

estadoFinal <- "Falto"

FinSi

Escribir "¡Datos registrados con exito en consola!"

Escribir "Estudiante: ", nombre

Escribir "Fecha: ", fecha

Escribir "Estado: ", estadoFinal

2:

Escribir " "

Escribir "=== CONTENIDO DEL HISTORIAL ==="

Escribir "Mostrando registros almacenados..."

3:

Escribir "Saliendo del programa..."

De Otro Modo:

Escribir "Opcion incorrecta."

FinSegun

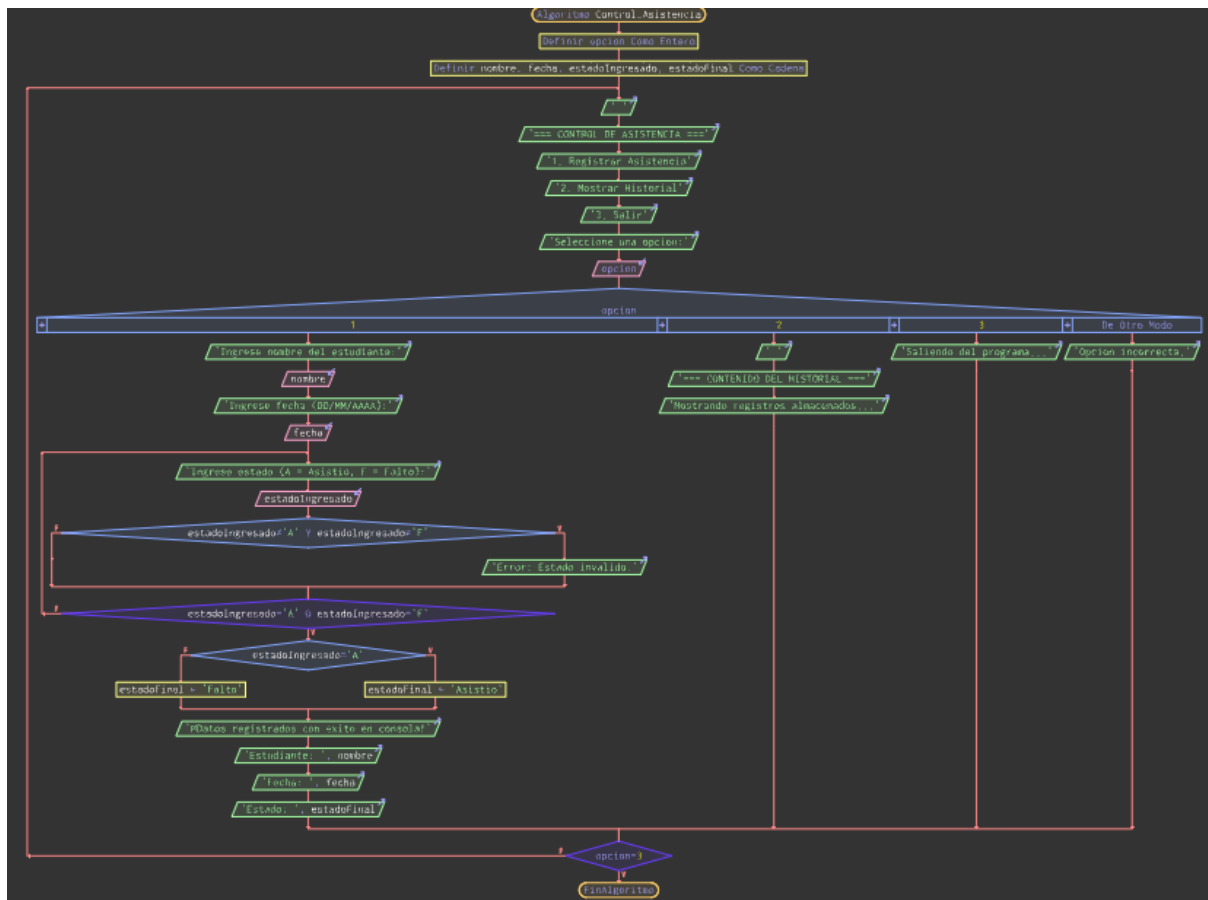
Hasta Que opcion = 3

FinAlgoritmo

5.Diagrama de Flujo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



6.Codigo en C++

```
#include <iostream>
```

```
#include <string>
```

```
#include <fstream>
```

```
using namespace std;
```

```
int main() {
```

```
    int opcion;
```

```
    string nombre, fecha, estadoIngresado, estadoFinal, lineaLeida;
```

```
    do {
```

```
        cout << "=== CONTROL DE ASISTENCIA ===\n";
```

```
        cout << "1. Registrar Asistencia\n";
```

```
        cout << "2. Mostrar Historial y Contar Faltas\n";
```

```
        cout << "3. Salir\n";
```

```
        cout << "Seleccione una opcion: ";
```

```
        cin >> opcion;
```

```
        cin.ignore(); // Limpiar el buffer de entrada para evitar saltos con getline
```

```
    switch (opcion) {
```

```
        case 1:
```

```
            cout << "Ingrese nombre del estudiante: ";
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
getline(cin, nombre);
cout << "Ingrese fecha (DD/MM/AAAA): ";
getline(cin, fecha);

// Validación estricta con ciclo Do-While
do {
    cout << "Ingrese estado (A = Asistio, F = Falto): ";
    getline(cin, estadoIngresado);
    if (estadoIngresado != "A" && estadoIngresado != "F") {
        cout << "Error: Por favor ingrese unicamente 'A' o 'F'.\\n";
    }
} while (estadoIngresado != "A" && estadoIngresado != "F");

// Asignar palabra completa según la entrada validada
if (estadoIngresado == "A") {
    estadoFinal = "Asistio";
} else {
    estadoFinal = "Falto";
}

// Guardar de forma secuencial en el archivo de texto
{
    ofstream archivoEscritura("asistencia.txt", ios::app);
    if (archivoEscritura.is_open()) {
        archivoEscritura << nombre << " | " << fecha << " | " << estadoFinal <<
        "\\n";

        archivoEscritura.close();
        cout << "¡Asistencia registrada con éxito!\\n";
    } else {
        cout << "Error al abrir el archivo para guardar.\\n";
    }
}
break;

case 2: {
    int totalFaltas = 0;
    ifstream archivoLectura("asistencia.txt");
    cout << "=== HISTORIAL DE ASISTENCIA ===\\n";

    // Leer el archivo línea por línea
    if (archivoLectura.is_open()) {
        while (getline(archivoLectura, lineaLeida)) {
            cout << lineaLeida << endl;
            // Contar de manera acumulativa si la línea contiene "Falto"
```



```
        if (lineaLeida.find("Falto") != string::npos) {
            totalFaltas++;
        }
    }
    archivoLectura.close();
    cout << "Total de faltas acumuladas: " << totalFaltas << endl;
} else {
    cout << "No hay registros guardados aun.\n";
}
break;
}

case 3:
    cout << "Saliendo del sistema...\n";
    break;

default:
    cout << "Opcion invalida, intente de nuevo.\n";
}
} while (opcion != 3);

return 0;
}
```

7.Codigo en Java

```
import java.util.Scanner;
import java.io.FileWriter;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        int opcion;
        String nombre, fecha, estadoIngresado, estadoFinal;

        do {
            System.out.println("=== CONTROL DE ASISTENCIA ===");
            System.out.println("1. Registrar Asistencia");
            System.out.println("2. Mostrar Historial y Contar Faltas");
            System.out.println("3. Salir");
            System.out.print("Seleccione una opcion: ");
            opcion = teclado.nextInt();
            teclado.nextLine(); // Limpiar el buffer del salto de línea
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
switch (opcion) {
    case 1:
        System.out.print("Ingrese nombre del estudiante: ");
        nombre = teclado.nextLine();
        System.out.print("Ingrese fecha (DD/MM/AAAA): ");
        fecha = teclado.nextLine();

        // Validación de datos utilizando ciclo repetitivo
        do {
            System.out.print("Ingrese estado (A = Asistio, F = Falto): ");
            estadoIngresado = teclado.nextLine().toUpperCase();
            if (!estadoIngresado.equals("A") && !estadoIngresado.equals("F")) {
                System.out.println("Error: Ingrese un estado valido ('A' o 'F').");
            }
        } while (!estadoIngresado.equals("A") && !estadoIngresado.equals("F"));

        if (estadoIngresado.equals("A")) {
            estadoFinal = "Asistio";
        } else {
            estadoFinal = "Falto";
        }

        // Escritura persistente en archivo .txt
        try (FileWriter fw = new FileWriter("asistencia.txt", true)) {
            fw.write(nombre + " | " + fecha + " | " + estadoFinal + "\n");
            System.out.println("¡Asistencia registrada con éxito!");
        } catch (IOException e) {
            System.out.println("Ocurrió un error al escribir en el archivo.");
        }
        break;

    case 2:
        int totalFaltas = 0;
        System.out.println("=== HISTORIAL DE ASISTENCIA ===");

        // Lectura secuencial y visualización del historial
        try (FileReader fr = new FileReader("asistencia.txt");
            BufferedReader br = new BufferedReader(fr)) {

            String lineaLeida;
            while ((lineaLeida = br.readLine()) != null) {
                System.out.println(lineaLeida);
                // Conteo acumulativo de faltas encontradas
            }
        }
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        if (lineaLeida.contains("Falto")) {
            totalFaltas++;
        }
    }
    System.out.println("Total de faltas acumuladas: " + totalFaltas);

    } catch (IOException e) {
        System.out.println("No hay registros de asistencia guardados aun.");
    }
    break;

case 3:
    System.out.println("Saliendo del sistema...");
    break;

default:
    System.out.println("Opcion no valida, intente de nuevo.");
}
} while (opcion != 3);

teclado.close();
}
}
```

8.Prueba de Solución en C++

```
=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 1
Ingrese nombre del estudiante: Mateo
Ingrese fecha (DD/MM/AAAA): 22-05-2026
Ingrese estado (A = Asistio, F = Falto): A
¡Asistencia registrada con exito!

=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 2

=== HISTORIAL DE ASISTENCIA ===
Mateo | 22-05-2026 | Asistio

Total de faltas acumuladas: 0

=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 1
Ingrese nombre del estudiante: Jhair
Ingrese fecha (DD/MM/AAAA): 13-05-2026
Ingrese estado (A = Asistio, F = Falto): F
¡Asistencia registrada con exito!

=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 2

=== HISTORIAL DE ASISTENCIA ===
Mateo | 22-05-2026 | Asistio
Jhair | 13-05-2026 | Falto

Total de faltas acumuladas: 1
```




9.Prueba de Solución en Java

```
=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 1
Ingrese nombre del estudiante: Mateo
Ingrese fecha (DD/MM/AAAA): 22-05-2026
Ingrese estado (A = Asistio, F = Falto): A
Asistencia registrada con exito!

=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 1
Ingrese nombre del estudiante: Jhair
Ingrese fecha (DD/MM/AAAA): 13-05-2026
Ingrese estado (A = Asistio, F = Falto): F
Asistencia registrada con exito!

=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 2

=== HISTORIAL DE ASISTENCIA ===
Mateo | 22-05-2026 | Asistio
Mateo | 22-05-2026 | AsistioJhair | 13-05-2026 | Falto
Total de faltas acumuladas: 1
```



```
usuarios.txt  asistencia.txt X
Archivo  Editar  Ver

Ingrese nombre del estudiante: Mateo
Ingrese fecha (DD/MM/AAAA): 22-05-2026
Ingrese estado (A = Asistio, F = Falto): A
Asistencia registrada con exito!

=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 1
Ingrese nombre del estudiante: Jhair
Ingrese fecha (DD/MM/AAAA): 13-05-2026
Ingrese estado (A = Asistio, F = Falto): F
Asistencia registrada con exito!

=== CONTROL DE ASISTENCIA ===
1. Registrar Asistencia
2. Mostrar Historial y Contar Faltas
3. Salir
Seleccione una opcion: 2

=== HISTORIAL DE ASISTENCIA ===
Mateo | 22-05-2026 | Asistio
Jhair | 13-05-2026 | Falto
-----|

usuarios.txt  asistencia.txt X
Archivo  Editar  Ver

=== HISTORIAL DE ASISTENCIA ===
Mateo | 22-05-2026 | Asistio
Jhair | 13-05-2026 | Falto
```

Ejercicio 7

1. Análisis del Problema

Se requiere simular un cajero automático utilizando Programación Orientada a Objetos (POO) y persistencia de datos. El programa debe poder leer y actualizar un archivo (cajero.txt) que almacena el nombre del usuario y su saldo actual. Debe permitir realizar consultas, depósitos y retiros, asegurando que no se pueda retirar más dinero del disponible.

2. Variables de Entrada, Proceso y Salida

- **Variables de Entrada:** Datos ingresados por el usuario para navegar por el menú y definir los montos de cada transacción. (Ejemplo conceptual: int opcion; float monto; string usuario;).
- **Variables de Proceso:** Variables utilizadas para realizar las operaciones aritméticas que



actualizan la cuenta y objetos para manejar el archivo. (Ejemplo conceptual: saldo = saldo + monto; y manejadores de lectura/escritura para cajero.txt).

- **Variables de Salida:** Mensajes en consola que confirman las transacciones, alertan errores o muestran el estado de la cuenta (Ejemplo conceptual: cout << "Saldo actual: \$" << saldo;). Adicionalmente, la actualización física de los datos en el archivo de texto.

3. Algoritmo

1. Inicio.
2. Declarar e inicializar el objeto de la clase Cajero, asignando un nombre de usuario y el nombre del archivo de texto a manipular.
3. Abrir el archivo en modo de lectura para extraer el saldo inicial. Si el archivo no existe o está vacío, inicializar la variable de saldo con un valor inicial de 0.
4. Iniciar un ciclo iterativo (ej. do-while). Definir el menú principal para que el ciclo se repita y permita al usuario elegir acciones de forma continua.
5. Leer y almacenar la opción de entrada de tipo numérico seleccionada por el usuario.
6. Evaluar mediante una estructura condicional (ej. switch) la opción para dirigir el flujo hacia la consulta, depósito o retiro.
7. Si la opción es depositar: leer la variable monto, verificar mediante una condición que sea mayor a 0, sumarla a la variable saldo y guardar inmediatamente el nuevo registro en el archivo de texto.
8. Si la opción es retirar: leer la variable monto y evaluar mediante una condición compuesta si es mayor a 0 y menor o igual al saldo actual.
9. Asignar la resta al saldo si la condición de retiro se cumple, guardando el cambio en el archivo. Si no se cumple, mostrar una alerta de fondos insuficientes.
10. Mostrar en pantalla los datos actualizados (saldo final) o los mensajes de estado actuales para verificar el proceso.
11. Finalizar el ciclo iterativo principal cuando se ingrese la opción específica de salida.
12. Fin.

4. Pseudocódigo

Algoritmo CajeroAutomatico

Definir opcion Como Entero

Definir monto, saldo Como Real

Definir usuario Como Cadena

usuario <- "Alexander"

saldo <- 0.0

// Se llama con paréntesis obligatorios en modo estricto

CargarDatos(usuario, saldo)

Repetir

Escribir ""

Escribir "--- CAJERO AUTOMÁTICO ---"

Escribir "1. Consultar Saldo"

Escribir "2. Depositar Dinero"

Escribir "3. Retirar Dinero"

Escribir "4. Salir"

Escribir "Seleccione una opción: "

Leer opcion

Segun opcion Hacer

1:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Usuario: ", usuario
Escribir "Saldo actual: \$", saldo

2:

Escribir "Ingrese el monto a depositar:"
Leer monto
Si monto > 0 Entonces
saldo <- saldo + monto
GuardarDatos(usuario, saldo)
Escribir "Depósito exitoso."

Sino

Escribir "Monto inválido."

FinSi

3:

Escribir "Ingrese el monto a retirar:"
Leer monto
Si monto > 0 Y monto <= saldo Entonces
saldo <- saldo - monto
GuardarDatos(usuario, saldo)
Escribir "Retiro exitoso."

Sino

Escribir "Fondos insuficientes o monto inválido."

FinSi

4:

Escribir "Saliendo del sistema..."

De Otro Modo:

Escribir "Opción no válida."

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

// Definición explícita de subprocesos abajo del algoritmo principal

SubProceso CargarDatos(usuario Por Referencia, saldo Por Referencia)

Escribir "Abriendo archivo cajero.txt en modo lectura..."

Escribir "Datos cargados correctamente."

FinSubProceso

SubProceso GuardarDatos(usuario, saldo)

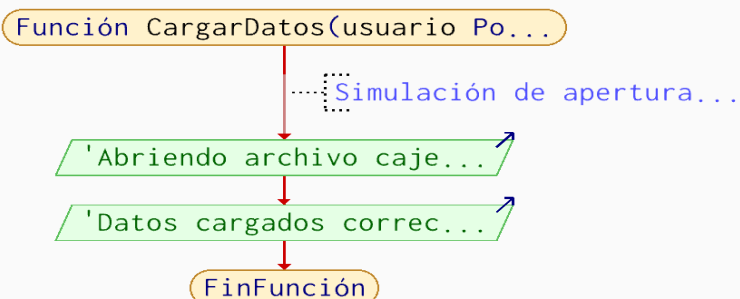
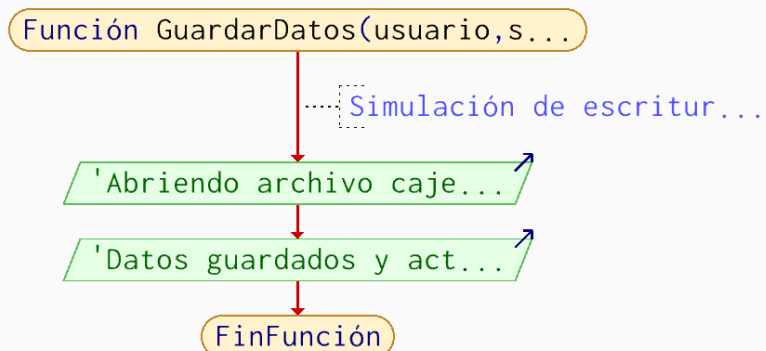
Escribir "Abriendo archivo cajero.txt en modo escritura..."

Escribir "Datos guardados y actualizados físicamente."

FinSubProceso

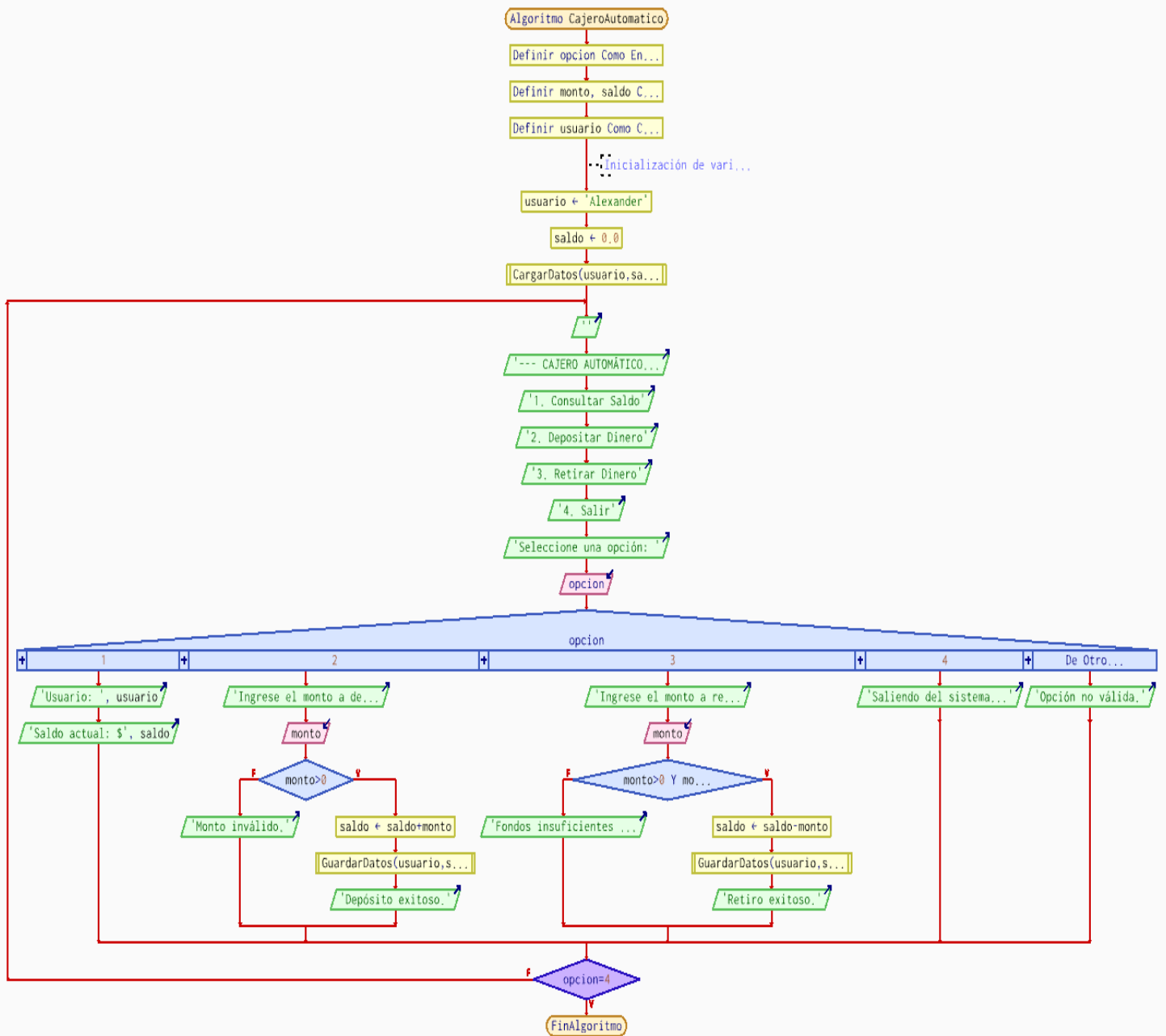


5. Diagrama de flujo





UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



6. Código c++

```
#include <iostream>
#include <fstream>
#include <string>
```

```
using namespace std;
```

```
class Cajero {
private:
    string usuario;
    float saldo;
    string nombreArchivo;

    void cargarDatos() {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
ifstream archivo(nombreArchivo.c_str());
if (archivo.is_open()) {
    archivo >> usuario >> saldo;
    archivo.close();
} else {
    // Valores por defecto si no existe el archivo
    usuario = "Estudiante";
    saldo = 0.0;
}

void guardarDatos() {
    ofstream archivo(nombreArchivo.c_str());
    archivo << usuario << endl;
    archivo << saldo << endl;
    archivo.close();
}

public:
    Cajero(string u, string archivo) {
        usuario = u;
        nombreArchivo = archivo;
        cargarDatos();
    }

    void consultarSaldo() {
        cout << "Usuario: " << usuario << " | Saldo actual: $" << saldo << endl;
    }

    void depositar(float monto) {
        if (monto > 0) {
            saldo += monto;
            guardarDatos();
            cout << "Deposito exitoso." << endl;
        } else {
            cout << "Monto invalido." << endl;
        }
    }

    void retirar(float monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            guardarDatos();
            cout << "Retiro exitoso." << endl;
        } else {
            cout << "Fondos insuficientes o monto invalido." << endl;
        }
    }
};

int main() {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Cajero miCajero("Alexander", "cajero.txt");
int opcion;
float monto;

do {
    cout << "\n--- Cajero Automatico ---" << endl;
    cout << "1. Consultar saldo" << endl;
    cout << "2. Depositar dinero" << endl;
    cout << "3. Retirar dinero" << endl;
    cout << "4. Salir" << endl;
    cout << "Ingrese una opcion: ";
    cin >> opcion;

    switch (opcion) {
        case 1:
            miCajero.consultarSaldo();
            break;
        case 2:
            cout << "Ingrese monto a depositar: ";
            cin >> monto;
            miCajero.depositar(monto);
            break;
        case 3:
            cout << "Ingrese monto a retirar: ";
            cin >> monto;
            miCajero.retirar(monto);
            break;
        case 4:
            cout << "Saliendo..." << endl;
            break;
        default:
            cout << "Opcion no valida." << endl;
    }
} while (opcion != 4);

return 0;
}
```

Pruebas de solución



Imagen n y n pruebaa C++ en visual Code

```
--- Cajero Automatico ---
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Ingrese una opcion: 1
Usuario: Estudiante | Saldo actual: $0

--- Cajero Automatico ---
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Ingrese una opcion: 2
Ingrese monto a depositar: 100
Deposito exitoso.

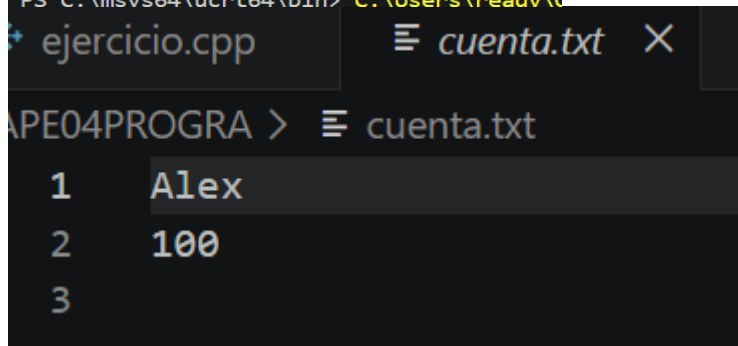
--- Cajero Automatico ---
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Ingrese una opcion: 1
Usuario: Estudiante | Saldo actual: $100

--- Cajero Automatico ---
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Ingrese una opcion: 4
Saliendo...
PS C:\msvs64\ucrt64\bin> C:\Users\readv\
```

```
--- Cajero Automatico ---
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Ingrese una opcion: 1
Usuario: Estudiante | Saldo actual: $100

--- Cajero Automatico ---
1. Consultar saldo
2. Depositar dinero
3. Retirar dinero
4. Salir
Ingrese una opcion: 4
Saliendo...
PS C:\msys64\ucrt64\bin> |
```

Imagen n Archivo.txt. donde se guardan los datos



7. Codigo Java

```
import java.io.*;
import java.util.Scanner;

class Cajero {
    private String usuario;
    private double saldo;
    private String nombreArchivo = "cajeroj.txt";

    public Cajero(String usuario) {
        this.usuario = usuario;
        cargarDatos();
    }
}
```



```
private void cargarDatos() {
    try {
        File archivo = new File(nombreArchivo);
        if (archivo.exists()) {
            BufferedReader br = new BufferedReader(new FileReader(archivo));
            this.usuario = br.readLine();
            this.saldo = Double.parseDouble(br.readLine());
            br.close();
        } else {
            this.saldo = 0.0;
        }
    } catch (Exception e) {
        System.out.println("Error al cargar datos.");
    }
}

private void guardarDatos() {
    try {
        BufferedWriter bw = new BufferedWriter(new FileWriter(nombreArchivo));
        bw.write(this.usuario + "\n");
        bw.write(this.saldo + "\n");
        bw.close();
    } catch (Exception e) {
        System.out.println("Error al guardar datos.");
    }
}

public void consultarSaldo() {
    System.out.println("Usuario: " + usuario + " | Saldo: $" + saldo);
}

public void depositar(double monto) {
    if (monto > 0) {
        saldo += monto;
        guardarDatos();
        System.out.println("Depósito exitoso.");
    }
}

public void retirar(double monto) {
    if (monto > 0 && monto <= saldo) {
        saldo -= monto;
        guardarDatos();
        System.out.println("Retiro exitoso.");
    } else {
        System.out.println("Fondos insuficientes.");
    }
}

public class cajerosim {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Ingrese nombre de usuario: ");
    String nombre = scanner.nextLine();
    Cajero miCajero = new Cajero(nombre);
    int opcion;

    do {
        System.out.println("\n--- Cajero ---");
        System.out.println("1. Consultar 2. Depositar 3. Retirar 4. Salir");
        System.out.print("Opcion: ");
        opcion = scanner.nextInt();

        switch (opcion) {
            case 1: miCajero.consultarSaldo(); break;
            case 2:
                System.out.print("Monto a depositar: ");
                miCajero.depositar(scanner.nextDouble());
                break;
            case 3:
                System.out.print("Monto a retirar: ");
                miCajero.retirar(scanner.nextDouble());
                break;
        }
    } while (opcion != 4);
    scanner.close();
}
```

Prueba de solución

```
Ingrese nombre de usuario: Alexander

--- Cajero ---
1. Consultar 2. Depositar 3. Retirar 4. Salir
Opcion: 1
Usuario: Alexander | Saldo: $0.0

--- Cajero ---
1. Consultar 2. Depositar 3. Retirar 4. Salir
Opcion: 2
Monto a depositar: 200
Depósito exitoso.

--- Cajero ---
1. Consultar 2. Depositar 3. Retirar 4. Salir
Opcion: 3
Monto a retirar: 10
Retiro exitoso.

--- Cajero ---
1. Consultar 2. Depositar 3. Retirar 4. Salir
Opcion: 1
Usuario: Alexander | Saldo: $190.0

--- Cajero ---
1. Consultar 2. Depositar 3. Retirar 4. Salir
Opcion: 4
PS C:\Users\ready\OneDrive\Documents\PROGRAMACION_UTA\.vscode>
```

Imagen n y n prueba de solución y archivo txt. Donde se guardan los datos

```
cajerosim.java  Estudiante.cla
cajeroj.txt
1 Alexander
2 190.0
3
```



Ejercicio 8

Se necesita un sistema para registrar libros en un archivo de texto. Se deben almacenar datos como código, título, autor y estado (Disponible/Prestado). Además, el programa debe permitir buscar libros específicos y filtrar listas según su estado leyendo directamente desde el archivo

1. Variables de Entrada, Proceso y Salida

- **Variables de Entrada:**
Datos de texto numéricos ingresados por el usuario para registrar un nuevo libro en el catálogo o navegar por el menú. (Ejemplo conceptual: string codigo, titulo, autor, estado; int opcion;).
- **Variables de Proceso:**
Variables utilizadas para evaluar condiciones durante las búsquedas y objetos para manejar el archivo. (Ejemplo conceptual: bool encontrado; string estadoBuscado; y manejadores de lectura/escritura para biblioteca.txt).
- **Variables de Salida:**
Mensajes en consola que muestran la lista de libros filtrados que cumplen con la condición de búsqueda (Ejemplo conceptual: cout << titulo << " por " << autor;). Adicionalmente, la escritura de los nuevos registros al final del archivo físico.

2. Algoritmo

1. Inicio.
2. Declarar e inicializar el objeto de la clase Biblioteca asignando el nombre del archivo de texto correspondiente como objetivo de persistencia.
3. Iniciar un ciclo iterativo (ej. do-while) para desplegar un menú de opciones interactivo que mantenga el programa en ejecución.
4. Leer y almacenar el dato de entrada tipo numérico para la opción elegida.
5. Evaluar mediante una estructura condicional (ej. switch) el camino de ejecución a seguir: registrar, ver disponibles o ver prestados.
6. Si la acción es registrar: leer y almacenar los datos de entrada tipo texto (codigo, titulo, autor y estado). Abrir el archivo en modo de adición (append) para insertar estos datos al final del mismo sin borrar los anteriores.
7. Si la acción es visualización filtrada: abrir el archivo estrictamente en modo de lectura.
8. Iniciar un ciclo iterativo de lectura (ej. while) que se repita mientras no se llegue al final del archivo, leyendo secuencialmente los datos de cada línea.
9. Evaluar mediante una estructura condicional si el valor de la variable estado extraída de la línea actual coincide exactamente con el valor de la variable estadoBuscado.
10. Mostrar en pantalla los datos extraídos del libro si la condición de estado se cumple, cambiando la bandera de la variable lógica a verdadero para confirmar que hubo resultados.
11. Cerrar los flujos de comunicación con el archivo de texto para liberar memoria, independientemente de la operación realizada.
12. Finalizar el ciclo iterativo principal si el usuario selecciona la opción de salida.
13. Fin.

3. Pseudocódigo

Algoritmo BibliotecaVirtual

Definir opcion Como Entero

Repetir

 Escribir ""

 Escribir "---- BIBLIOTECA VIRTUAL ----"

 Escribir "1. Registrar Libro"



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "2. Mostrar Libros Disponibles"

Escribir "3. Mostrar Libros Prestados"

Escribir "4. Salir"

Escribir "Seleccione una opción: "

Leer opcion

Segun opcion Hacer

1:

RegistrarLibro

2:

MostrarPorEstado("Disponible")

3:

MostrarPorEstado("Prestado")

4:

Escribir "Saliendo de la biblioteca..."

De Otro Modo:

Escribir "Opción no válida."

FinSegun

Hasta Que opcion = 4

FinAlgoritmo

SubProceso RegistrarLibro

Definir codigo, titulo, autor, estado Como Cadena

Escribir "Ingrese Código del libro:"

Leer codigo

Escribir "Ingrese Título del libro:"

Leer titulo

Escribir "Ingrese Autor del libro:"

Leer autor

Escribir "Ingrese Estado (Disponible/Prestado):"

Leer estado

Escribir "Abriendo archivo biblioteca.txt en modo adición..."

Escribir "Registro guardado al final del archivo con éxito."

FinSubProceso

SubProceso MostrarPorEstado(estadoBuscado)

Escribir ""

Escribir "--- LIBROS ", estadoBuscado, "S ---"

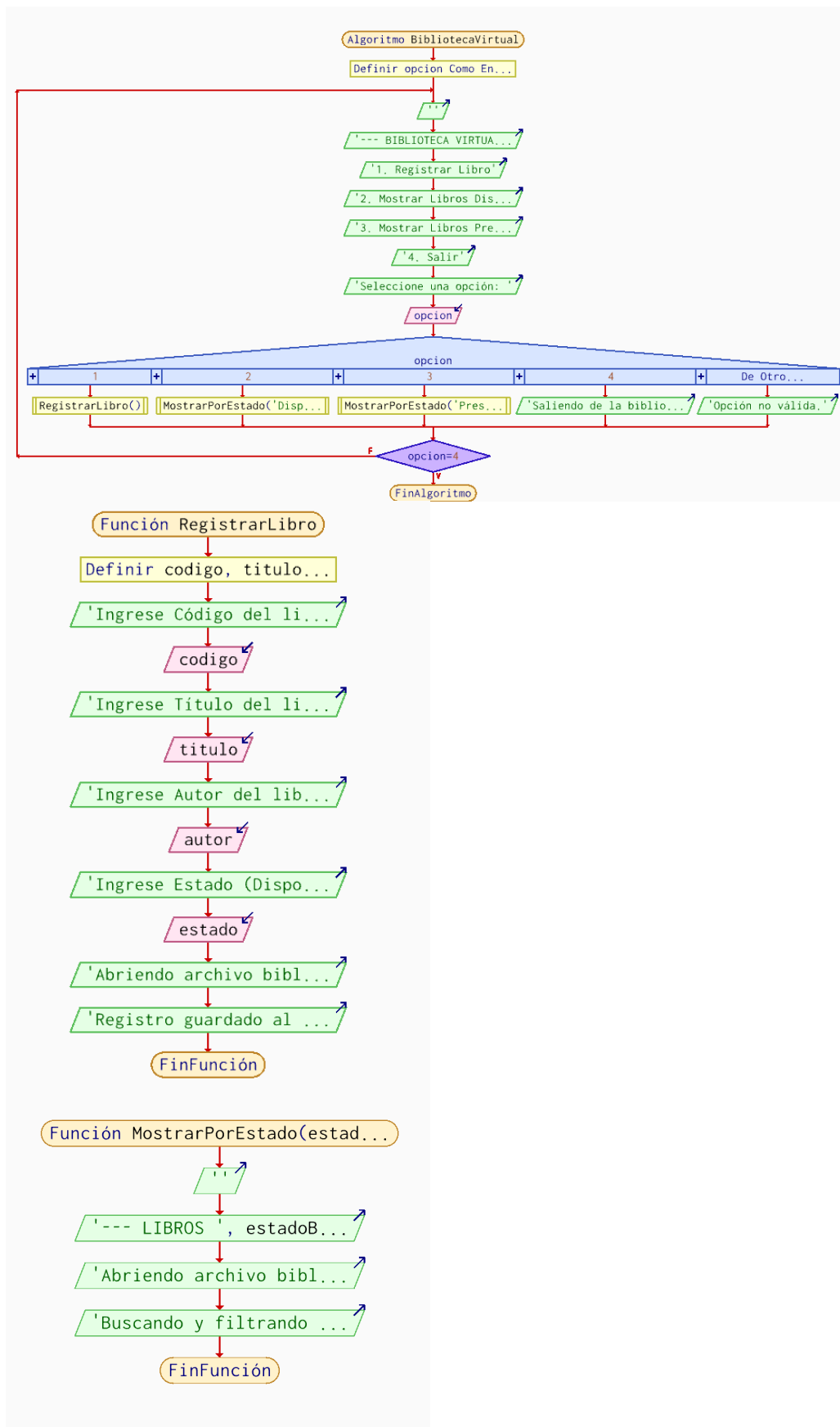
Escribir "Abriendo archivo biblioteca.txt en modo lectura..."

Escribir "Buscando y filtrando registros secuencialmente..."

FinSubProceso



4. Diagrama de flujo





5. Codigo C++

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

class Biblioteca {
private:
    string nombreArchivo;

public:
    Biblioteca(string archivo) {
        nombreArchivo = archivo;
        // Asegura que el archivo exista creándolo si no está
        ofstream archivoVerificar(nombreArchivo.c_str(), ios::app);
        archivoVerificar.close();
    }

    void registrarLibro() {
        string codigo, titulo, autor, estado;

        cout << "\n--- Registrar Nuevo Libro ---" << endl;
        cout << "IngreseCodigo: "; cin >> codigo;
        cout << "IngreseTitulo (Use-guiones-sin-espacios): "; cin >> titulo;
        cout << "IngreseAutor (Use-guiones-sin-espacios): "; cin >> autor;
        cout << "Estado (Disponible/Prestado): "; cin >> estado;

        ofstream archivo(nombreArchivo.c_str(), ios::app);
        if (archivo.is_open()) {
            archivo << codigo << " " << titulo << " " << autor << " " << estado << endl;
            archivo.close();
            cout << "-> Libro registrado exitosamente en el archivo." << endl;
        } else {
            cout << "-> Error al abrir el archivo para escribir." << endl;
        }
    }

    void mostrarPorEstado(string estadoBuscado) {
        ifstream archivo(nombreArchivo.c_str());
        string codigo, titulo, autor, estado;
        bool encontrado = false;

        cout << "\n===== " << endl;
        cout << "    LIBROS " << estadoBuscado << "S" << endl;
        cout << "===== " << endl;

        if (archivo.is_open()) {
            // Lee fila por fila mapeando las columnas del archivo txt
            while (archivo >> codigo >> titulo >> autor >> estado) {
                if (estado == estadoBuscado) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        cout << "ID: " << codigo << " | " << titulo << " - Autor: " << autor << endl;
        encontrado = true;
    }
}
archivo.close();

if (!encontrado) {
    cout << "No se encontraron libros con el estado: " << estadoBuscado <<
endl;
}
} else {
    cout << "Error al abrir el archivo para lectura." << endl;
}
cout << "=====" << endl;
}
};

int main() {
    Biblioteca miBiblioteca("biblioteca.txt");
    int opcion;

    do {
        cout << "\n--- MENU BIBLIOTECA VIRTUAL ---" << endl;
        cout << "1. Registrar Libro" << endl;
        cout << "2. Mostrar Libros Disponibles" << endl;
        cout << "3. Mostrar Libros Prestados" << endl;
        cout << "4. Salir" << endl;
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                miBiblioteca.registrarLibro();
                break;
            case 2:
                miBiblioteca.mostrarPorEstado("Disponible");
                break;
            case 3:
                miBiblioteca.mostrarPorEstado("Prestado");
                break;
            case 4:
                cout << "Saliendo del sistema de biblioteca..." << endl;
                break;
            default:
                cout << "Opcion invalida. Intente de nuevo." << endl;
        }
    } while (opcion != 4);

    return 0;
}
```

Pruebas de solución



Imagen n y n prueba solución y texto txt hecha en visual

```
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Mostrar Libros Disponibles
3. Mostrar Libros Prestados
4. Salir
Seleccione una opcion: 1

--- Registrar Nuevo Libro ---
IngreseCodigo: 121212
IngreseTitulo (Use-guiones-sin-espacios): Don-Quijote
IngreseAutor (Use-guiones-sin-espacios): Miguel-Cervantes
Estado (Disponible/Prestado): Disponible
-> Libro registrado exitosamente en el archivo.

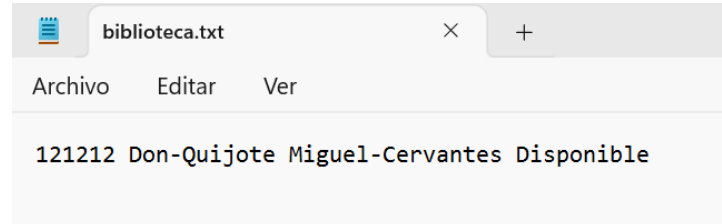
--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Mostrar Libros Disponibles
3. Mostrar Libros Prestados
4. Salir
Seleccione una opcion: 2

=====
LIBROS Disponibles
=====
ID: 121212 | Don-Quijote - Autor: Miguel-Cervantes
=====

3. Mostrar Libros Prestados
4. Salir
Seleccione una opcion: 3

=====
LIBROS Prestados
=====
No se encontraron libros con el estado: Prestado
=====

--- MENU BIBLIOTECA VIRTUAL ---
1. Registrar Libro
2. Mostrar Libros Disponibles
3. Mostrar Libros Prestados
4. Salir
Seleccione una opcion: 4
Saliendo del sistema de biblioteca...
PS C:\msvs64\ucrt64\bin>
```



6. Código Java

```
import java.io.*;
import java.util.Scanner;
```

```
class Biblioteca {
    private String archivo = "biblioteca.txt";
```

```
    public void registrarLibro(Scanner scanner) {
        System.out.print("Código: ");
        String cod = scanner.nextLine().trim();
        System.out.print("Título: ");
        String tit = scanner.nextLine().trim();
        System.out.print("Autor: ");
        String aut = scanner.nextLine().trim();
        System.out.print("Estado (Disponible/Prestado): ");
        String est = scanner.nextLine().trim();
```

```
        if (!est.equalsIgnoreCase("Disponible") && !est.equalsIgnoreCase("Prestado")) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        System.out.println("Estado inválido. Usa Disponible o Prestado.");
        return;
    }

    est = est.substring(0, 1).toUpperCase() + est.substring(1).toLowerCase();

    try {
        BufferedWriter bw = new BufferedWriter(new FileWriter(archivo, true));
        bw.write(cod + " " + tit + " " + aut + " " + est + "\n");
        bw.close();
        System.out.println("Libro guardado.");
    } catch (IOException e) {
        System.out.println("Error al guardar: " + e.getMessage());
    }
}

public void mostrarPorEstado(String estadoBuscado) {
    try {
        BufferedReader br = new BufferedReader(new FileReader(archivo));
        String linea;
        System.out.println("\n--- Libros " + estadoBuscado + "s ---");
        while ((linea = br.readLine()) != null) {
            if (linea.contains(estadoBuscado)) {
                System.out.println(linea);
            }
        }
        br.close();
    } catch (IOException e) {
        System.out.println("No se pudo leer el archivo (puede que esté vacío).");
    }
}

public class MainBiblioteca {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Biblioteca biblio = new Biblioteca();
        int opcion = 0;

        do {
            System.out.println("\n--- Biblioteca ---");
            System.out.println("1. Registrar 2. Disponibles 3. Prestados 4. Salir");
            System.out.print("Opción: ");
            String entrada = scanner.nextLine().trim();

            try {
                opcion = Integer.parseInt(entrada);
            } catch (NumberFormatException e) {
                System.out.println("Por favor ingresa un número válido.");
                continue;
            }
        }
```



```
switch (opcion) {  
    case 1:  
        biblio.registrarLibro(scanner);  
        break;  
    case 2:  
        biblio.mostrarPorEstado("Disponible");  
        break;  
    case 3:  
        biblio.mostrarPorEstado("Prestado");  
        break;  
    case 4:  
        System.out.println("Saliendo...");  
        break;  
    default:  
        System.out.println("Opción no válida.");  
}  
} while (opcion != 4);  
scanner.close();  
}  
}
```

Prueba de solución

Imagen n y n pruebas de solución y archivo guardado hecha en Visual Code

```
--- Biblioteca ---  
1. Registrar 2. Disponibles 3. Prestados 4. Salir  
Opción: 2  
No se pudo leer el archivo (puede que esté vacío).  
  
--- Biblioteca ---  
1. Registrar 2. Disponibles 3. Prestados 4. Salir  
Opción: 1  
Código: 111111  
Título: Las 48 leyes del poder  
Autor: Anonimo  
Estado (Disponible/Prestado): Disponible  
Libro guardado.  
  
--- Biblioteca ---  
1. Registrar 2. Disponibles 3. Prestados 4. Salir  
Opción: 2  
  
--- Libros Disponibles ---  
111111 Las 48 leyes del poder Anonimo Disponible  
  
--- Biblioteca ---  
1. Registrar 2. Disponibles 3. Prestados 4. Salir  
Opción: 3  
  
--- Libros Prestados ---  
  
--- Biblioteca ---  
1. Registrar 2. Disponibles 3. Prestados 4. Salir  
Opción: 4  
Saliendo...
```

≡ biblioteca.txt

```
1 111111 Las 48 leyes del poder Anonimo Disponible  
2
```



Ejercicio 9

1. Analisis

El objetivo es desarrollar un sistema para registrar ventas, calcular totales, generar una factura en pantalla y guardar el historial en un archivo de texto utilizando POO

Incluye entradas con Nombre del producto (texto), cantidad (entero mayor a 0) y precio (decimal mayor a 0)

Procesos:

1. Validar que la cantidad y el precio sean números positivos.
2. Calcular el total (cantidad * precio)
3. Instanciar un objeto de la clase Venta
4. Guardar los datos de la venta en ventas.txt

En la salidas estara Menú interactivo, impresión de la factura actual en consola y lectura/visualización del historial desde el archivo .txt

2. Declaracion de variables

Entrada:

opcion: int - Opción elegida por el usuario en el menú interactivo

producto: string - Nombre del producto ingresado por consola

cantidad: int - Cantidad de unidades vendidas (debe ser > 0)

precio: double - Precio unitario del producto (debe ser > 0)

Proceso:

sistema / app: Objeto - Instancia de la clase principal para manejar la lógica POO

nuevaVenta: Objeto - Instancia de la clase Venta creada para procesar los datos

archivo / fw: File / FileWriter - Descriptor u objeto para abrir y escribir en ventas.txt

br / lector: BufferedReader - Objeto para leer el flujo de datos del archivo de texto

Salida:

total: double - Resultado de multiplicar cantidad por precio

linea: string / String - Texto leído desde el archivo que se muestra en el historial

3. Algoritmo

1.- Inicio.

2.- Mostrar el menú interactivo: (1. Registrar Venta, 2. Ver Historial, 3. Salir).

3.- Leer y validar la opcion.

4.- Si opcion es 1:

4.1.- Solicitar producto.

4.2.- Solicitar cantidad (validar que sea > 0).

4.3.- Solicitar precio (validar que sea > 0).

4.4.-Calcular total = cantidad * precio.

4.5.- Mostrar factura en pantalla.

4.6.- Abrir archivo ventas.txt en modo adición (append) y escribir los datos.

5.- Si opcion es 2:

5.1.- Abrir archivo ventas.txt en modo lectura.

5.2.-Leer línea por línea e imprimir en pantalla. Si no existe, avisar al usuario.

6.- Si opcion es 3:

6.1.- Fin. (Salir del programa).

7.- Si no es 3, volver al paso 2.

8.- Fin



4. Código en PSeInt

Algoritmo SistemaDeVentas

Definir opcion Como Entero;
Definir producto Como Caracter;
Definir cantidad Como Entero;
Definir precio, total Como Real;

Repetir

Escribir "=== MENU DE VENTAS ===";
Escribir "1. Registrar Venta";
Escribir "2. Ver Historial (Simulado)";
Escribir "3. Salir";
Escribir "Seleccione una opcion: ";
Leer opcion;

Segun opcion Hacer

1:

Escribir "Ingrese nombre del producto: ";
Leer producto;

// validacion cantidad

Repetir

Escribir "Ingrese cantidad (> 0): ";
Leer cantidad;
Si (cantidad <= 0) Entonces
Escribir "Error: La cantidad debe ser mayor a 0.";
FinSi
Hasta Que (cantidad > 0);

// validacion precio

Repetir

Escribir "Ingrese precio (> 0): ";
Leer precio;
Si (precio <= 0) Entonces
Escribir "Error: El precio debe ser mayor a 0.";
FinSi
Hasta Que (precio > 0);

// calculos de la operacion

*total = cantidad * precio;*

// impresion de factura

Escribir "--- FACTURA ---";
Escribir "Producto: ", producto;
Escribir "Cantidad: ", cantidad;
Escribir "Precio: \$", precio;



Escribir "Total a pagar: \$", total;
Escribir "-----";

2:

Escribir "En PSeInt estandar no se leen archivos .txt nativamente.";
Escribir "Revise las versiones de C++ y Java para esta funcion.";

3:

Escribir "Saliendo del sistema...";

De Otro Modo:

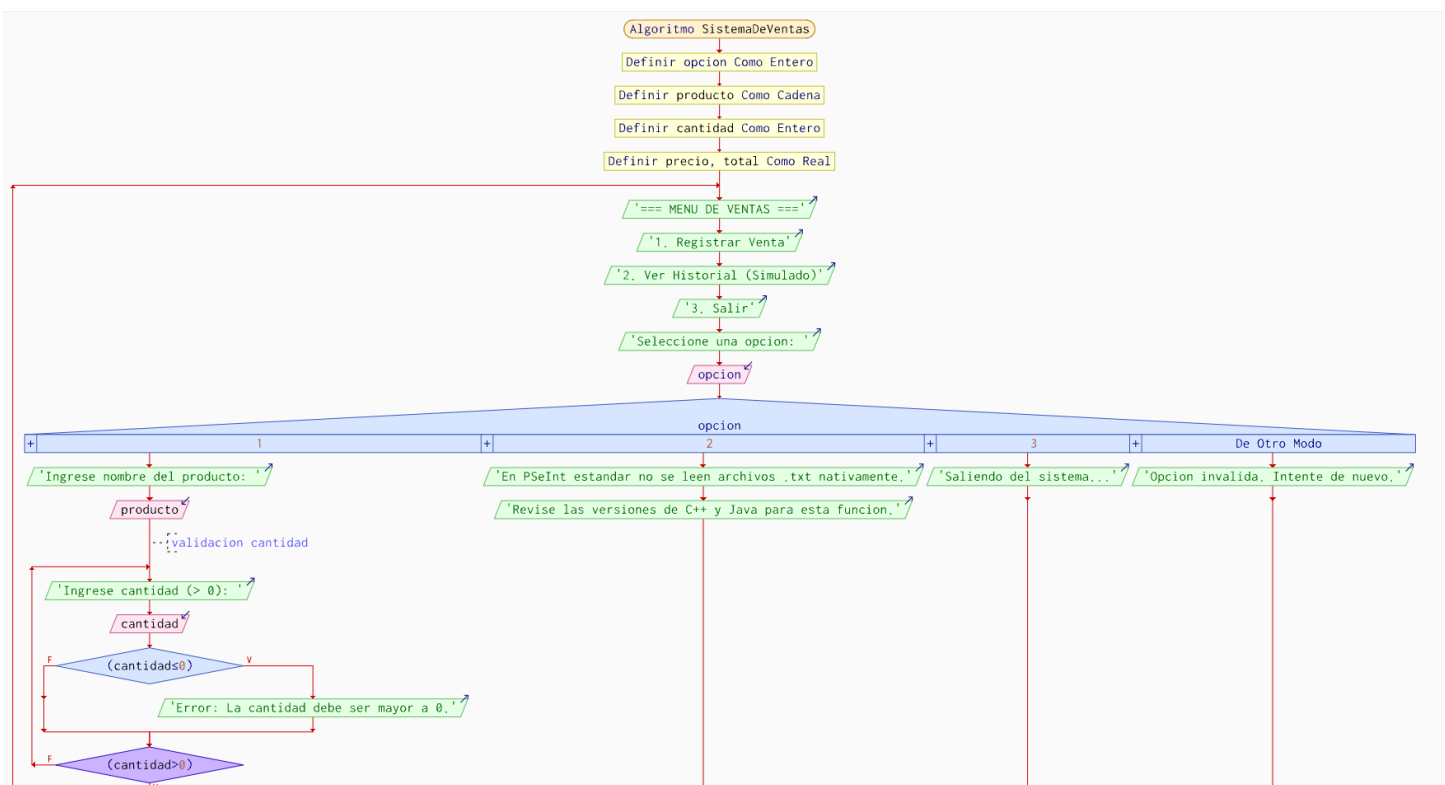
Escribir "Opcion invalida. Intente de nuevo.";

FinSegun

Hasta Que (opcion = 3);

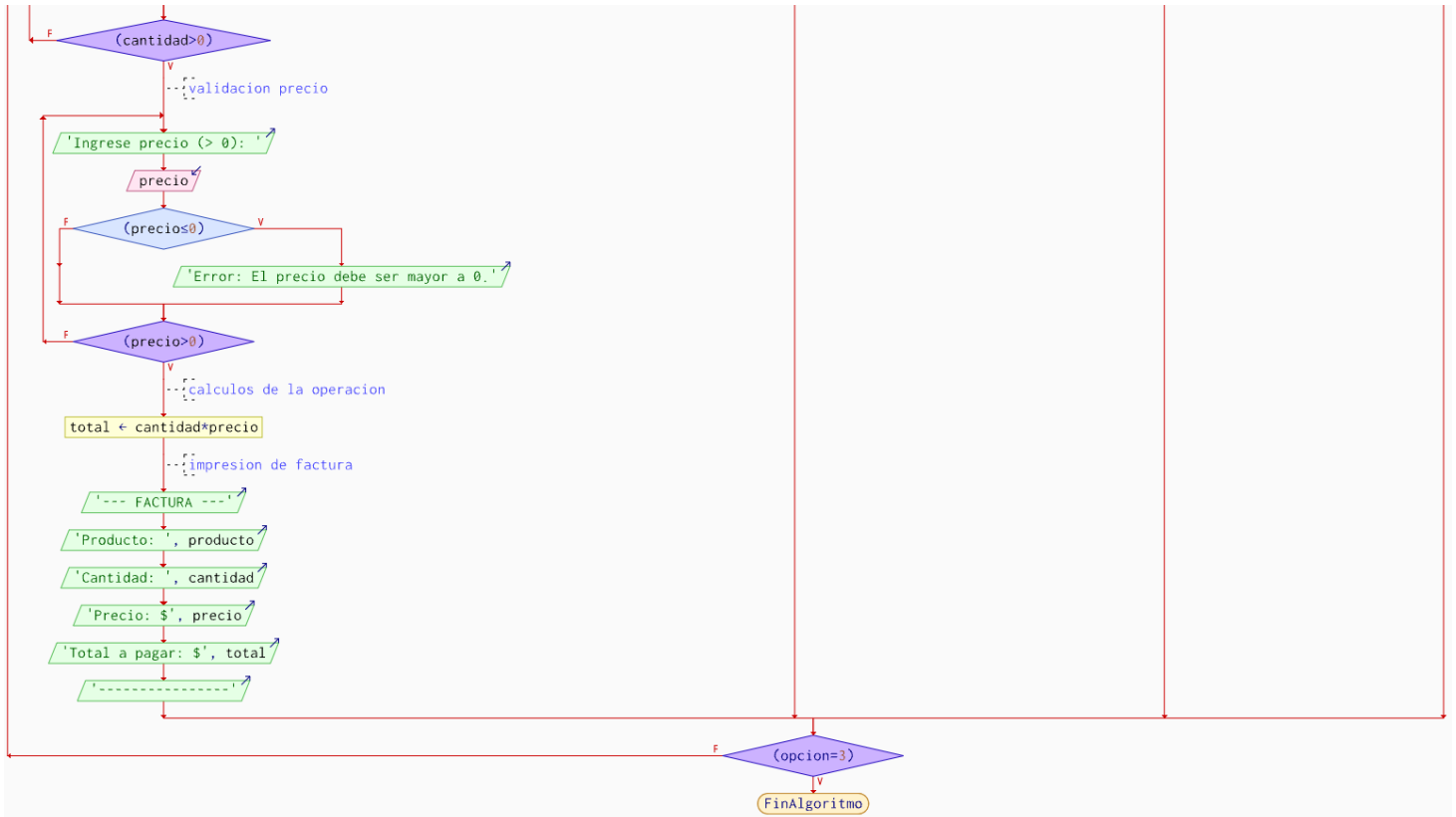
FinAlgoritmo

5. Diagrama de flujo





UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



6. Código en C++

```
#include <iostream>
#include <fstream>
#include <string>
#include <limits>
```

```
using namespace std;
```

```
// clase Venta
```

```
class Venta {
```

```
private:
```

```
    string producto;
```

```
    int cantidad;
```

```
    double precio;
```

```
    double total;
```

```
public:
```

```
    // constructor
```

```
    Venta(string prod, int cant, double prec) {
```

```
        producto = prod;
```

```
        cantidad = cant;
```

```
        precio = prec;
```

```
        calcularTotal();
```

```
    }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// metodo calcular el total
void calcularTotal() {
    total = cantidad * precio;
}

// metodo para generar y mostrar la factura
void generarFactura() {
    cout << "\n--- FACTURA ---" << endl;
    cout << "Producto: " << producto << endl;
    cout << "Cantidad: " << cantidad << endl;
    cout << "Precio un.: $" << precio << endl;
    cout << "Total: $" << total << endl;
    cout << "-----" << endl;
}

// metodo guardar en archivo .txt
void guardarEnArchivo() {
    ofstream archivo("ventas.txt", ios::app); // Abrir en modo append
    if (archivo.is_open()) {
        archivo << producto << " | Cant: " << cantidad << " | Precio: $" << precio << "
| Total: $" << total << "\n";
        archivo.close();
        cout << "Venta guardada exitosamente en ventas.txt\n";
    } else {
        cout << "Error al abrir el archivo.\n";
    }
}

};

// Clase Gestor
class SistemaVentas {
public:
    // registrar y validar datos
    void registrarVenta() {
        string prod;
        int cant;
        double prec;

        cout << "\nIngrese nombre del producto: ";
        cin.ignore();
        getline(cin, prod);

        // validación de cantidad
        cout << "Ingrese cantidad: ";
        while (!(cin >> cant) || cant <= 0) {
            cout << "Error. Ingrese una cantidad valida (> 0): ";
            cin.clear();
        }
    }
};
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

// validación de precio
cout << "Ingrese precio: ";
while (!(cin >> prec) || prec <= 0) {
    cout << "Error. Ingrese un precio valido (> 0): ";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

// instanciar objeto
Venta nuevaVenta(prod, cant, prec);
nuevaVenta.generarFactura();
nuevaVenta.guardarEnArchivo();
}

// mostrar historial leyendo el .txt
void mostrarHistorial() {
    string linea;
    ifstream archivo("ventas.txt");

    cout << "\n=== HISTORIAL DE VENTAS ===" << endl;
    if (archivo.is_open()) {
        while (getline(archivo, linea)) {
            cout << linea << endl;
        }
        archivo.close();
    } else {
        cout << "No hay ventas registradas o no se pudo abrir el archivo." << endl;
    }
    cout << "=====\n";
}

// Menu
void iniciarMenu() {
    int opcion;
    do {
        cout << "\n1. Registrar Venta\n2. Ver Historial\n3. Salir\nElija opcion: ";
        if (!(cin >> opcion)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            opcion = 0;
        }

        switch (opcion) {
            case 1: registrarVenta(); break;
```



```
case 2: mostrarHistorial(); break;
case 3: cout << "Saliendo...\n"; break;
default: cout << "Opcion invalida.\n";
}
} while (opcion != 3);
}
};

int main() {
    SistemaVentas sistema;
    sistema.iniciarMenu();
    return 0;
}
```

7. Prueba de escritorio en C++

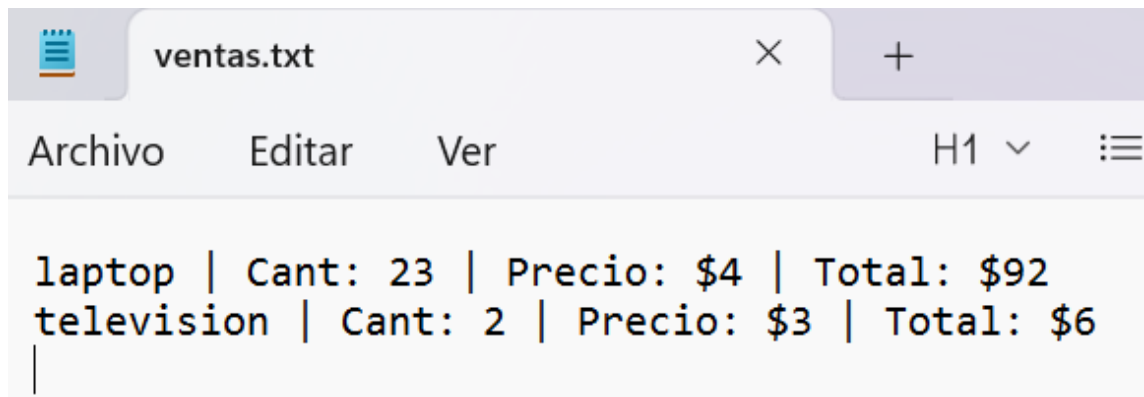
```
1. Registrar Venta
2. Ver Historial
3. Salir
Elija opcion: 1

Ingrese nombre del producto: laptop
Ingrese cantidad: er
Error. Ingrese una cantidad valida (> 0): 23
Ingrese precio: 1
Error. Ingrese un precio valido (> 0): 4

--- FACTURA ---
Producto: laptop
Cantidad: 23
Precio un.: $4
Total: $92
-----
```

```
1. Registrar Venta
2. Ver Historial
3. Salir
Elija opcion: 2

=== HISTORIAL DE VENTAS ===
laptop | Cant: 23 | Precio: $4 | Total: $92
television | Cant: 2 | Precio: $3 | Total: $6
=====
```



8. Codigo en Java

```
import java.io.*;
import java.util.Scanner;

// clase Venta
class Venta {
    private String producto;
    private int cantidad;
    private double precio;
    private double total;

    // constructor
    public Venta(String producto, int cantidad, double precio) {
        this.producto = producto;
        this.cantidad = cantidad;
        this.precio = precio;
        calcularTotal();
    }

    // Método para calcular total
    private void calcularTotal() {
        this.total = this.cantidad * this.precio;
    }

    // metodo generar factura
    public void generarFactura() {
        System.out.println("\n--- FACTURA ---");
        System.out.println("Producto: " + producto);
        System.out.println("Cantidad: " + cantidad);
        System.out.println("Precio un.: $" + precio);
        System.out.println("Total: $" + total);
        System.out.println("-----");
    }

    // metodo guardar en txt
    public void guardarEnArchivo() {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
try (FileWriter fw = new FileWriter("ventas.txt", true);
    PrintWriter pw = new PrintWriter(fw)) {
    pw.println(producto + " | Cant: " + cantidad + " | Precio: $" + precio + " |
Total: $" + total);
    System.out.println("Venta guardada exitosamente en ventas.txt");
} catch (IOException e) {
    System.out.println("Error al guardar en el archivo: " + e.getMessage());
}
}
}

// clase Principal y de Gestión
public class SistemaVentas {
    private Scanner scanner;

    public SistemaVentas() {
        scanner = new Scanner(System.in);
    }

    // metodo para registrar venta validando
    public void registrarVenta() {
        System.out.print("\nIngrese nombre del producto: ");
        String prod = scanner.nextLine();

        int cant = 0;
        double prec = 0;

        // validacion cantidad
        while (true) {
            System.out.print("Ingrese cantidad (> 0): ");
            try {
                cant = Integer.parseInt(scanner.nextLine());
                if (cant > 0) break;
                System.out.println("Error: La cantidad debe ser positiva.");
            } catch (NumberFormatException e) {
                System.out.println("Error: Ingrese un número entero válido.");
            }
        }

        // validacion precio
        while (true) {
            System.out.print("Ingrese precio (> 0): ");
            try {
                prec = Double.parseDouble(scanner.nextLine());
                if (prec > 0) break;
                System.out.println("Error: El precio debe ser positivo.");
            } catch (NumberFormatException e) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        System.out.println("Error: Ingrese un número decimal válido.");
    }
}

// crear objeto y ejecutar métodos
Venta nuevaVenta = new Venta(prod, cant, prec);
nuevaVenta.generarFactura();
nuevaVenta.guardarEnArchivo();
}

// metodo para mostrar historial
public void mostrarHistorial() {
    System.out.println("\n=== HISTORIAL DE VENTAS ===");
    File archivo = new File("ventas.txt");
    if (archivo.exists()) {
        try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error al leer el archivo.");
        }
    } else {
        System.out.println("No hay ventas registradas.");
    }
    System.out.println("=====");
}

// Menu
public void iniciarMenu() {
    int opcion = 0;
    do {
        System.out.println("\n1. Registrar Venta");
        System.out.println("2. Ver Historial");
        System.out.println("3. Salir");
        System.out.print("Elija opción: ");

        try {
            opcion = Integer.parseInt(scanner.nextLine());
        } catch (NumberFormatException e) {
            opcion = 0;
        }

        switch (opcion) {
            case 1: registrarVenta(); break;
            case 2: mostrarHistorial(); break;
```



```
        case 3: System.out.println("Saliendo..."); break;
        default: System.out.println("Opción inválida.");
    }
} while (opcion != 3);
}

public static void main(String[] args) {
    SistemaVentas sistema = new SistemaVentas();
    sistema.iniciarMenu();
}
}
```

9. Prueba de escritorio en Java

```
1. Registrar Venta
2. Ver Historial
3. Salir
Elija opción: 1

Ingrese nombre del producto: carro
Ingrese cantidad (> 0): 3
Ingrese precio (> 0): 2

--- FACTURA ---
Producto: carro
Cantidad: 3
Precio un.: $2.0
Total: $6.0
-----
```

```
1. Registrar Venta
2. Ver Historial
3. Salir
Elija opción: 2

=== HISTORIAL DE VENTAS ===
carro | Cant: 3 | Precio: $2.0 | Total: $6.0
mueble | Cant: 1 | Precio: $231.0 | Total: $231.0
=====
```



```
ventas.txt
Archivo  Editar  Ver

carro | Cant: 3 | Precio: $2.0 | Total: $6.0
mueble | Cant: 1 | Precio: $231.0 | Total: $231.0
```

Ejercicio 10

1. Analisis

El objetivo es desarrollar un sistema de gestión (CRUD: Crear, Leer, Actualizar, Eliminar) para un registro de Usuarios/Clientes aplicando POO, métodos, validaciones y almacenamiento en un archivo .txt

En las entradas: datos del usuario (ID/Cédula, Nombre, Edad) y la opción del menú seleccionada

En procesos:

- Validar que la edad y el ID sean correctos, instanciar el objeto y agregarlo al archivo .txt (modo adición)
- Abrir el archivo .txt y mostrar todos los registros iterando línea por línea
- Cargar los datos del .txt en memoria, buscar el ID, modificar los atributos solicitados y reescribir todo el archivo
- Cargar los datos, omitir el ID especificado y reescribir el archivo sin ese registro

En las salidas el menú interactivo, confirmaciones de éxito/error y visualización de la lista de usuarios

2. Declaracion de variables

Entrada:

opcion: int - Opción seleccionada del menú (1 al 5)

idIngresado / id: string / String - ID o cédula del usuario a registrar o buscar

nombreIngresado: string / String - Nombre del usuario que se va a guardar o actualizar

edadIngresada: int - Edad del usuario (validada como > 0)

buscarId: string / String - ID clave que digita el usuario para modificar o eliminar.



Proceso:

listaUsuarios / lista: vector / ArrayList - Estructura dinámica en memoria para contener los objetos provisorios

encontrado: bool / boolean - Bandera (flag) para saber si el ID buscado existe en el sistema

i: int - Contador / Índice utilizado para recorrer los bucles

archivo / ss: fstream / stringstream - Manejadores de archivos y formatos de conversión de texto a objeto.

Salida:

u: Objeto Usuario - Instancia utilizada para imprimir los datos estructurados de los registros recuperados.

3. Algoritmo

- 1.- Inicio
- 2.- Mostrar el Menú Interactivo (1. Crear, 2. Leer, 3. Actualizar, 4. Eliminar, 5. Salir)
- 3.-Leer opcion
- 4.- Si opcion = 1 (Crear): Solicitar id, nombre y edad. Validar edad > 0. Guardar en usuarios.txt
- 5.- Si opcion = 2 (Leer): Abrir usuarios.txt, leer e imprimir cada línea
- 6.- Si opcion = 3 (Actualizar): Leer usuarios.txt, buscar por id. Si existe, pedir nuevos datos, modificar y reescribir el archivo completo
- 7.- Si opcion = 4 (Eliminar): Leer usuarios.txt, buscar por id. Reescribir el archivo omitiendo el registro que coincida con el id
- 8.- Si opcion = 5: Finalizar programa
- 9.- Si no es 5, regresar al paso 2
- 10.- Fin

4. Código en PSeInt

Algoritmo CRUD_Usuarios

Definir MAX, totalRegistros, opcion Como Entero;



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Definir ids, nombres, idBuscar Como Cadena;

Definir edades, i, j, nuevaEdad Como Entero;

Definir encontrado Como Logico;

MAX <- 100;

totalRegistros <- 0;

Dimension ids[100], nombres[100], edades[100];

Repetir

 Escribir "";

 Escribir "=== SISTEMA CRUD ===";

 Escribir "Desarrollador: Alejandro";

 Escribir "1. Crear";

 Escribir "2. Leer";

 Escribir "3. Actualizar";

 Escribir "4. Eliminar";

 Escribir "5. Salir";

 Escribir "Opcion: ";

 Leer opcion;

Segun opcion Hacer

 1:

 Si totalRegistros < MAX Entonces

 Escribir "Ingrese ID: ";

 Leer ids[totalRegistros + 1];

 Escribir "Ingrese Nombre: ";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Leer nombres[totalRegistros + 1];

Repetir

 Escribir "Ingrese Edad (>0): ";

 Leer nuevaEdad;

 Si nuevaEdad <= 0 Entonces

 Escribir "Error: La edad debe ser mayor a 0.";

 FinSi

 Hasta Que nuevaEdad > 0

 edades[totalRegistros + 1] <- nuevaEdad;

 totalRegistros <- totalRegistros + 1;

 Escribir "Usuario creado.";

Sino

 Escribir "Error: Memoria llena.";

FinSi

2:

Si totalRegistros = 0 Entonces

 Escribir "No hay registros.";

Sino

 Escribir "";

 Escribir "--- LISTA DE USUARIOS ---";

 Para i <- 1 Hasta totalRegistros Con Paso 1 Hacer

 Escribir "ID: ", ids[i], " | Nombre: ", nombres[i], " | Edad: ", edades[i];

 FinPara

FinSi



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



3:

Si totalRegistros = 0 Entonces

Escribir "No hay registros para actualizar.";

Sino

Escribir "Ingrese ID a actualizar: ";

Leer idBuscar;

encontrado <- Falso;

Para i <- 1 Hasta totalRegistros Con Paso 1 Hacer

Si ids[i] = idBuscar Entonces

Escribir "Nuevo Nombre: ";

Leer nombres[i];

Repetir

Escribir "Nueva Edad (>0): ";

Leer nuevaEdad;

Si nuevaEdad <= 0 Entonces

Escribir "Error: La edad debe ser mayor a 0.";

FinSi

Hasta Que nuevaEdad > 0

edades[i] <- nuevaEdad;

encontrado <- Verdadero;

Escribir "Usuario actualizado.";

FinSi

FinPara

Si encontrado = Falso Entonces



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "ID no encontrado.";

FinSi

FinSi

4:

Si totalRegistros = 0 Entonces

Escribir "No hay registros para eliminar.";

Sino

Escribir "Ingrese ID a eliminar: ";

Leer idBuscar;

encontrado <- Falso;

Para i <- 1 Hasta totalRegistros Con Paso 1 Hacer

Si ids[i] = idBuscar Entonces

encontrado <- Verdadero;

Para j <- i Hasta totalRegistros - 1 Con Paso 1 Hacer

ids[j] <- ids[j + 1];

nombres[j] <- nombres[j + 1];

edades[j] <- edades[j + 1];

FinPara

totalRegistros <- totalRegistros - 1;

Escribir "Usuario eliminado.";

FinSi

FinPara

Si encontrado = Falso Entonces

Escribir "ID no encontrado.";

FinSi



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



FinSi

5:

Escribir "Saliendo...";

De Otro Modo:

Escribir "Opcion invalida.";

FinSegun

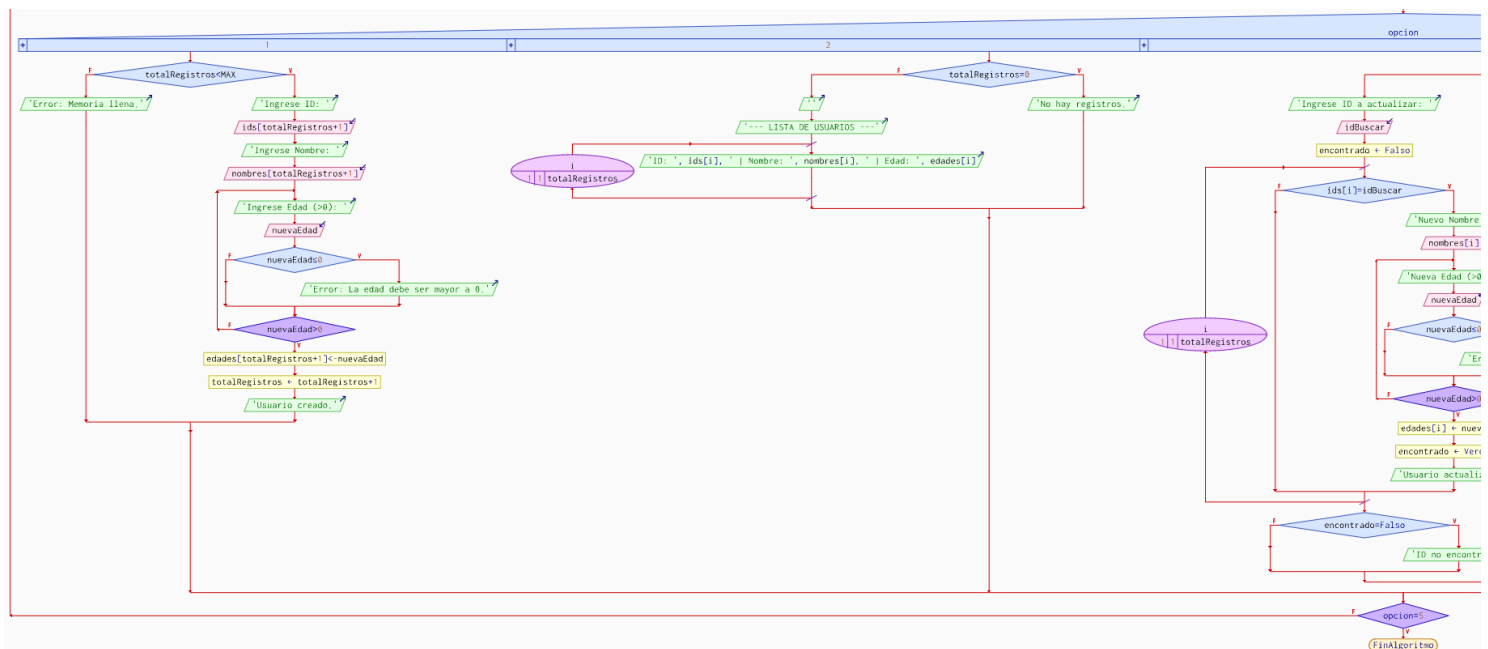
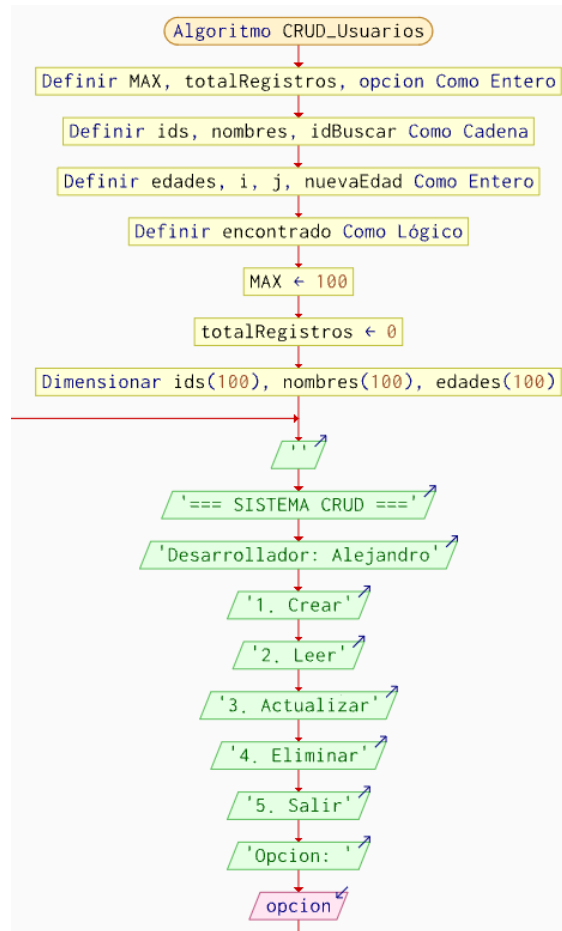
Hasta Que opcion = 5

FinAlgoritmo

5. Diagrama de flujo

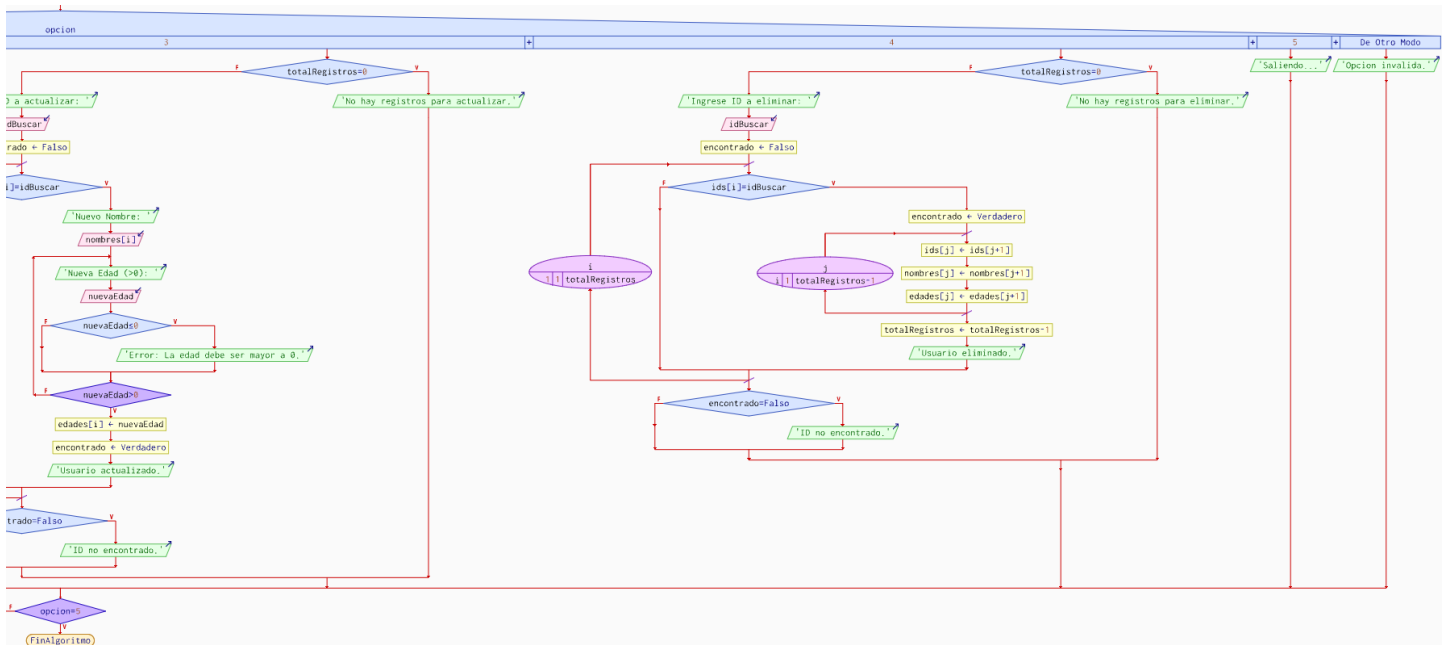


UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026





UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



6. Código en C++

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
```

```
using namespace std;
```

```
// Clase Entidad
```

```
class Usuario {
public:
    string id;
    string nombre;
    int edad;
```

```
    Usuario(string i, string n, int e) : id(i), nombre(n), edad(e) {}
};
```

```
// Clase Gestor
```

```
class GestorCRUD {
private:
```

```
    string nombreArchivo = "usuarios.txt";
```

```
    // metodo privado para cargar datos a memoria
```

```
    vector<Usuario> cargarDatos() {
        vector<Usuario> lista;
        ifstream archivo(nombreArchivo);
```



```
string linea, id, nombre, edadStr;

if (archivo.is_open()) {
    while (getline(archivo, linea)) {
        stringstream ss(linea);
        getline(ss, id, ',');
        getline(ss, nombre, ',');
        getline(ss, edadStr, ',');
        if (!id.empty()) {
            lista.push_back(Usuario(id, nombre, stoi(edadStr)));
        }
    }
    archivo.close();
}
return lista;
}

// metodo privado para sobrescribir el archivo
void guardarTodos(vector<Usuario> & lista) {
    ofstream archivo(nombreArchivo, ios::trunc); // trunc borra y reescribe
    for (auto & u : lista) {
        archivo << u.id << "," << u.nombre << "," << u.edad << "\n";
    }
    archivo.close();
}

public:
void crear() {
    string id, nombre;
    int edad;
    cout << "Ingrese ID: "; cin >> id;
    cin.ignore();
    cout << "Ingrese Nombre: "; getline(cin, nombre);

    do {
        cout << "Ingrese Edad (>0): "; cin >> edad;
    } while (edad <= 0);

    ofstream archivo(nombreArchivo, ios::app);
    archivo << id << "," << nombre << "," << edad << "\n";
    archivo.close();
    cout << "Usuario creado.\n";
}

void leer() {
    vector<Usuario> lista = cargarDatos();
    if (lista.empty()) {
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
cout << "No hay registros.\n";
return;
}
cout << "\n--- LISTA DE USUARIOS ---\n";
for (auto& u : lista) {
    cout << "ID: " << u.id << " | Nombre: " << u.nombre << " | Edad: " << u.edad
<< "\n";
}
}

void actualizar() {
    string idBuscar;
    cout << "Ingrese ID a actualizar: "; cin >> idBuscar;

    vector<Usuario> lista = cargarDatos();
    bool encontrado = false;

    for (auto& u : lista) {
        if (u.id == idBuscar) {
            cin.ignore();
            cout << "Nuevo Nombre: "; getline(cin, u.nombre);
            do {
                cout << "Nueva Edad (>0): "; cin >> u.edad;
            } while (u.edad <= 0);
            encontrado = true;
            break;
        }
    }

    if (encontrado) {
        guardarTodos(lista);
        cout << "Usuario actualizado.\n";
    } else {
        cout << "ID no encontrado.\n";
    }
}

void eliminar() {
    string idBuscar;
    cout << "Ingrese ID a eliminar: "; cin >> idBuscar;

    vector<Usuario> lista = cargarDatos();
    vector<Usuario> nuevaLista;
    bool encontrado = false;

    for (auto& u : lista) {
        if (u.id != idBuscar) {
```



```
nuevaLista.push_back(u);
} else {
    encontrado = true;
}
}

if (encontrado) {
    guardarTodos(nuevaLista);
    cout << "Usuario eliminado.\n";
} else {
    cout << "ID no encontrado.\n";
}
}
};

int main() {
    GestorCRUD gestor;
    int opcion;

    do {
        cout << "\n=== SISTEMA CRUD ===" << endl;
        cout << "Desarrollador: Alejandro" << endl;
        cout << "1. Crear\n2. Leer\n3. Actualizar\n4. Eliminar\n5. Salir\nOpcion: ";
        cin >> opcion;

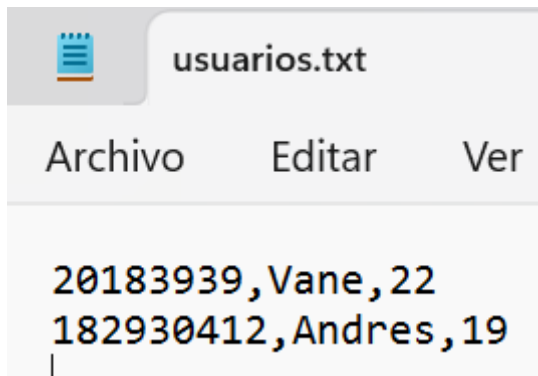
        switch (opcion) {
            case 1: gestor.crear(); break;
            case 2: gestor.leer(); break;
            case 3: gestor.actualizar(); break;
            case 4: gestor.eliminar(); break;
            case 5: cout << "Saliendo...\n"; break;
            default: cout << "Opcion invalida.\n";
        }
    } while (opcion != 5);

    return 0;
}
```



7. Prueba de escritorio en C++

```
--- LISTA DE USUARIOS ---  
ID: 20183939 | Nombre: Vane | Edad: 22  
ID: 182930412 | Nombre: Andres | Edad: 19  
  
=== SISTEMA CRUD ===  
Desarrollador: Alejandro  
1. Crear  
2. Leer  
3. Actualizar  
4. Eliminar  
5. Salir  
Opcion:
```



8. Código en Java

```
import java.io.*;  
import java.util.ArrayList;  
import java.util.Scanner;  
  
// clase entidad  
class Usuario {  
    String id;  
    String nombre;  
    int edad;  
  
    public Usuario(String id, String nombre, int edad) {  
        this.id = id;  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}
```



}

// clase principal y gestor CRUD

public class Main {

private static final String ARCHIVO = "usuarios.txt";

private Scanner scanner = new Scanner(System.in);

// metodo para leer archivo y pasarlo a memoria

private ArrayList<Usuario> cargarDatos() {

ArrayList<Usuario> lista = new ArrayList<>();

try (BufferedReader br = new BufferedReader(new FileReader(ARCHIVO))) {

String linea;

while ((linea = br.readLine()) != null) {

String[] datos = linea.split(",");

if (datos.length == 3) {

lista.add(new Usuario(datos[0], datos[1], Integer.parseInt(datos[2]));)

}

}

} catch (IOException e) {

}

return lista;

}

// metodo para reescribir todo el archivo

private void guardarTodos(ArrayList<Usuario> lista) {

try (PrintWriter pw = new PrintWriter(new FileWriter(ARCHIVO))) {

for (Usuario u : lista) {

pw.println(u.id + "," + u.nombre + "," + u.edad);

}

} catch (IOException e) {

System.out.println("Error al guardar archivo.");

}

}

public void crear() {

System.out.print("Ingreso ID: ");

String id = scanner.nextLine();

System.out.print("Ingreso Nombre: ");

String nombre = scanner.nextLine();

int edad = 0;

do {

System.out.print("Ingreso Edad (> 0): ");

try {

edad = Integer.parseInt(scanner.nextLine());

} catch (NumberFormatException e) {

edad = 0;



```
}  
} while (edad <= 0);  
  
// añadir al archivo  
try (PrintWriter pw = new PrintWriter(new FileWriter(ARCHIVO, true))) {  
    pw.println(id + "," + nombre + "," + edad);  
    System.out.println("Usuario guardado.");  
} catch (IOException e) {  
    System.out.println("Error de escritura.");  
}  
}  
  
public void leer() {  
    ArrayList<Usuario> lista = cargarDatos();  
    if (lista.isEmpty()) {  
        System.out.println("No hay registros.");  
        return;  
    }  
    System.out.println("\n--- LISTA DE USUARIOS ---");  
    for (Usuario u : lista) {  
        System.out.println("ID: " + u.id + " | Nombre: " + u.nombre + " | Edad: " +  
u.edad);  
    }  
}  
  
public void actualizar() {  
    System.out.print("Ingrese ID a actualizar: ");  
    String idBuscar = scanner.nextLine();  
  
    ArrayList<Usuario> lista = cargarDatos();  
    boolean encontrado = false;  
  
    for (Usuario u : lista) {  
        if (u.id.equals(idBuscar)) {  
            System.out.print("Nuevo Nombre: ");  
            u.nombre = scanner.nextLine();  
            do {  
                System.out.print("Nueva Edad (> 0): ");  
                u.edad = Integer.parseInt(scanner.nextLine());  
            } while (u.edad <= 0);  
            encontrado = true;  
            break;  
        }  
    }  
}  
  
if (encontrado) {  
    guardarTodos(lista);  
}
```



```
        System.out.println("Usuario actualizado exitosamente.");
    } else {
        System.out.println("ID no encontrado.");
    }
}

public void eliminar() {
    System.out.print("Ingrese ID a eliminar: ");
    String idBuscar = scanner.nextLine();

    ArrayList<Usuario> lista = cargarDatos();
    boolean eliminado = lista.removeIf(u -> u.id.equals(idBuscar));

    if (eliminado) {
        guardarTodos(lista);
        System.out.println("Usuario eliminado.");
    } else {
        System.out.println("ID no encontrado.");
    }
}

public void iniciar() {
    int opcion = 0;
    do {
        System.out.println("\n=== SISTEMA CRUD ===");
        System.out.println("Desarrollador: Alejandro");
        System.out.println("1. Crear");
        System.out.println("2. Leer");
        System.out.println("3. Actualizar");
        System.out.println("4. Eliminar");
        System.out.println("5. Salir");
        System.out.print("Opcion: ");

        try {
            opcion = Integer.parseInt(scanner.nextLine());
        } catch (Exception e) {
            opcion = 0;
        }

        switch (opcion) {
            case 1: crear(); break;
            case 2: leer(); break;
            case 3: actualizar(); break;
            case 4: eliminar(); break;
            case 5: System.out.println("Saliendo..."); break;
            default: System.out.println("Opcion invalida.");
        }
    }
```

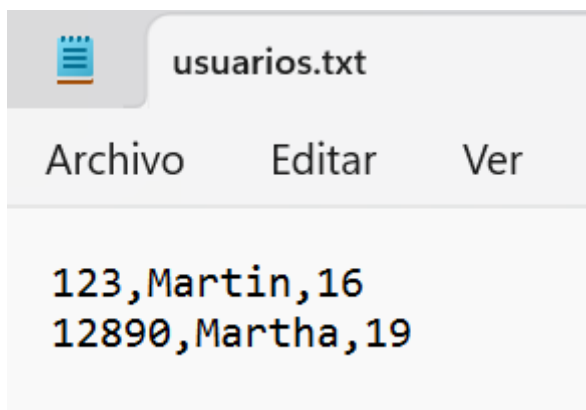


```
} while (opcion != 5);  
}
```

```
public static void main(String[] args) {  
    Main app = new Main();  
    app.iniciar();  
}  
}
```

9. Prueba de escritorio en Java

```
--- LISTA DE USUARIOS ---  
ID: 123 | Nombre: Martin | Edad: 16  
ID: 12890 | Nombre: Martha | Edad: 19  
  
=== SISTEMA CRUD ===  
Desarrollador: Alejandro  
1. Crear  
2. Leer  
3. Actualizar  
4. Eliminar  
5. Salir  
Opcion:
```



2.7 Resultados obtenidos

Durante el desarrollo de la guía práctica se implementaron exitosamente diez ejercicios que abarcan desde el registro básico de estudiantes hasta sistemas completo. En cada ejercicio se logró la persistencia de datos mediante archivos de texto (.txt), verificando que la información se almacene y recupere correctamente entre ejecuciones del programa. Los programas desarrollados en C++ y Java produjeron resultados equivalentes, confirmando la portabilidad de la lógica de negocio entre lenguajes. Se comprobó el funcionamiento correcto de las clases y sus métodos, la validación de entradas del usuario, el manejo de excepciones en operaciones de archivo y la actualización dinámica de registros sin pérdida de datos previamente almacenados. Las pruebas de solución realizadas evidencian que cada programa cumple con los requerimientos planteados en la guía.



Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.8 Conclusiones

- El uso de ficheros de texto como mecanismo de persistencia permite que los datos generados por un programa sobrevivan entre ejecuciones, constituyendo una base fundamental para comprender sistemas de almacenamiento más complejos como bases de datos relacionales.
- La Programación Orientada a Objetos facilita la organización del código al encapsular los datos y las operaciones sobre archivos dentro de clases, lo que mejora la legibilidad, el mantenimiento y la reutilización del software desarrollado.
- La implementación paralela en C++ y Java demuestra que los conceptos de manejo de ficheros son transversales a los lenguajes de programación, variando únicamente en la sintaxis y las bibliotecas empleadas, pero manteniendo la misma lógica estructural.
- El conocimiento adquirido en esta guía práctica constituye la base teórica y técnica necesaria para el desarrollo de un aplicativo web de ejercicios de programación, ya que un sistema de este tipo requiere: almacenar y recuperar teoría en archivos o bases de datos, gestionar los ejercicios (enunciados, pseudocódigos, código fuente) como registros estructurados, implementar operaciones CRUD para que los administradores puedan agregar, editar y eliminar contenido, y aplicar validaciones en el servidor similares a las realizadas en los ejercicios de consola.

2.9 Recomendaciones

- Se recomienda reemplazar el almacenamiento en archivos de texto plano por una base de datos relacional (como MySQL o PostgreSQL) en proyectos de mayor escala, ya que los archivos .txt no soportan búsquedas eficientes, concurrencia ni integridad referencial entre registros.
- Es importante implementar el manejo de excepciones en todas las operaciones de archivo, tanto en C++ como en Java, para evitar que errores de lectura o escritura provoquen el cierre inesperado del programa sin informar al usuario.
- Se sugiere documentar cada clase y método desarrollado utilizando comentarios descriptivos, facilitando así la comprensión del código por parte de otros desarrolladores o en revisiones futuras.
- Se aconseja practicar el modo de apertura de archivos (lectura, escritura, adición) con cuidado, verificando siempre que el archivo esté correctamente abierto antes de realizar operaciones, y cerrándolo de forma segura al finalizar para evitar pérdida de datos.

2.10 Referencias bibliográficas

[1] B. Stroustrup, El lenguaje de programación C++, 4.^a ed. Madrid, España: Addison-Wesley, 2013.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



-
- [2] H. M. Deitel y P. J. Deitel, Cómo programar en C++, 8.^a ed. México: Pearson Educación, 2012.
- [3] H. M. Deitel y P. J. Deitel, Cómo programar en Java, 10.^a ed. México: Pearson Educación, 2016.
- [4] Oracle Corporation, "Java SE Documentation – java.io package," Oracle, 2024. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/package-summary.html>. [Accedido: 22 may. 2025].